



North South University

Department of Electrical & Computer Engineering

Junior Design Project

XLSense: Intelligent Data Workflow & Spreadsheet Automation Platform

Submitted by: *Team Code XYZ*

Section: 15

Members:

Name	ID
Rubaiya Akter	2322072042
Adita Haq	2323037042
Farshid Abrar Labib (Section: 16)	2211809042

Faculty Advisor:

Name: Tanzilah Noor Shabnam

Designation: Senior Lecturer

ECE Department, North South University

Semester: Spring 2026

ACKNOWLEDGEMENTS

We, Rubaiya Akter, Adita Haq, Farshid Abrar Labib, would like to express our heartfelt gratitude towards our project supervisor, Tanzilah Noor Shabnam ma'am, from the Department of Electrical and Computer Engineering, North South University, Bangladesh, for her invaluable support, precise guidance, and constructive feedback throughout the course of this project.

Furthermore, we would like to thank the Department of Electrical and Computer Engineering, North South University, Bangladesh, for facilitating the project work and providing the necessary academic resources and laboratory facilities required for its successful completion.

Finally, we would also like to thank our families, friends, and loved ones for their countless sacrifices, unwavering encouragement, and continual support during every stage of this endeavor.

ABSTRACT

XLSense is an intelligent web-based platform designed to automate spreadsheet validation, formula auditing, and data quality analysis across multiple domain-specific contexts. Users upload raw spreadsheet files (.xlsx, .csv, .pdf, .txt), select a domain (Academic, HR, Finance, or Inventory), and receive processed, validated Excel files with auto-applied formulas, error detection, quality scoring, and AI-powered explanations. The system leverages Django 6.0 as the backend framework, MySQL for data persistence, pandas and openpyxl for data processing, and an OpenAI-compatible API for intelligent feature automation. Key contributions include domain-specific validation rule engines, real-time workbook editing with live cell validation, and AI-driven text-to-spreadsheet conversion for unstructured data sources. XLSense demonstrates practical full-stack development skills, API integration, data science tooling, and collaborative software engineering.

Keywords: Spreadsheet validation, Data quality, Formula auditing, AI explanations, Django, pandas, Domain-specific rules

TABLE OF CONTENTS

ACKNOWLEDGEMENTS.....	i
ABSTRACT.....	ii
TABLE OF CONTENTS.....	iii
LIST OF FIGURES.....	iv
LIST OF TABLES.....	v
Problem Statement.....	vi
Chapter 1: Introduction	1
1.1 Background and Motivation	1
1.2 Purpose and Goal of the Project.....	1
1.3 Organization of the Report.....	2
Chapter 2: Research Literature Review	3
2.1 Existing Research and Limitations	3
Chapter 3: Methodology.....	4
3.1 System Design	4
3.1a. Drawbacks of the Present System	4
3.1b. How XLSense Solves These Problems.....	5
3.1c. Implemented Features in Details.....	5
3.1d. Schematic Diagram / Flow Chart.....	7
3.2 Technology Used	11
Chapter 4: Investigation/Experiment, Result, Analysis, and Discussion.....	12
4.1 Methodology (Implementation Processes)	12
Chapter 5: Impacts of the Project	14
5.1 Societal, Health, Safety, Legal, and Cultural Issues	14
5.2 Environment and Sustainability.....	14
Chapter 6: Project Management	16
6.1 Project Planning and Budget.....	16
6.2 Week-wise Individual Member Contribution:	17
6.3 Pseudocode and algorithm:	23
6.4 Code Snippet.....	41
6.5 UI Snippet	74
6.6 Budget Planning:.....	84
Chapter 7: Conclusion	85
7.1 Summary	85
7.2 Limitations	85
7.3 Future Improvement.....	85
References	87

LIST OF FIGURES

Figure Name	Page
System Design	4
ER Diagram	7
Workflow	8
User Flow	9
Admin Flow	10
Validation Flow	10
Gantt Chart	16
UI Snippet - Homepage	43
UI Snippet - Login	43
UI Snippet - Sign up	44
UI Snippet - Dashboard	44
UI Snippet - Audit Logs (Admin only)	45
UI Snippet - Domain Templates	45
UI Snippet - Upload Page	46
UI Snippet - Workbook editor page	46-47
UI Snippet - Upload history:	47
UI Snippet - AI implementation in upload history	48
UI Snippet - AI Implementation in workbook editor	48

LIST OF TABLES

Figure Name	Page
How XLSense Solves These Problems (Chapter 3)	5
Technology Used	11
Testing Results	13
Week 1, 2 Contributions	17
Week 3, 4 Contributions	18
Week 5, 6 Contributions	19
Week 7 Contributions	20
Budget Planning	49

Problem Statement

Spreadsheet errors represent a significant and often underestimated source of data quality issues in modern organizations. Manual data validation is time-consuming, error-prone, and typically requires specialized Microsoft Excel expertise that many end users lack. Existing tools on the market either provide generic validation with no domain awareness, require expensive commercial licenses, or demand significant manual configuration.

The problem is compounded when organizations receive unstructured data from external sources, which are PDFs, text files, and scanned documents. That must be manually transcribed into structured spreadsheets before any validation can occur. This process is repetitive, error-prone, and does not scale.

Additionally, the lack of plain-language explanations for formula errors or validation failures creates a barrier for non-technical users who need to understand why their data was flagged and how to correct it. Commercial tools like Microsoft Excel provide limited built-in validation but do not offer AI-powered explanations or domain-specific rule sets for industries such as academia, human resources, finance, or inventory management.

XLSense addresses these problems by providing a unified, open-source web platform that automates spreadsheet validation, formula auditing, data quality scoring, and AI-powered explanations, all within a user-friendly interface accessible to non-technical users.

Chapter 1: Introduction

1.1 Background and Motivation

Spreadsheets remain one of the most widely used data management tools across industries. According to a 2023 survey by Wakefield Research, 81% of business professionals use spreadsheets for data management, and 67% report that spreadsheet errors have led to significant business consequences [1]. A landmark study by Raymond Panko found that 88% of all spreadsheets contain at least one formula error [2].

Despite the prevalence of spreadsheet usage, few tools exist that provide automated, domain-specific validation without requiring extensive manual configuration. Generic validators apply the same rules across all domains, ignoring industry-specific requirements such as grade boundaries in academia, salary caps in HR, budget thresholds in finance, or stock limits in inventory management.

The motivation for XLSense stems from three key observations:

Observation 1: Domain-Specific Rules Are Essential. A grade validation rule (0–100 for exam scores) differs fundamentally from a salary validation rule (minimum wage to maximum budget). Existing tools apply generic rules that cannot distinguish between domains.

Observation 2: AI Can Bridge the Expertise Gap. Non-technical users struggle to understand Excel formula errors. An AI-powered explanation system can translate technical error messages into plain language, helping users correct errors without needing Excel expertise.

Observation 3: Unstructured Data Needs Automation. Converting a PDF table or a raw text file into a structured, validated spreadsheet is currently a manual process. AI-driven text-to-spreadsheet conversion can automate this entirely.

1.2 Purpose and Goal of the Project

The purpose of XLSense is to build a comprehensive, domain-aware spreadsheet validation and automation platform that:

1. Accepts multiple file formats (.xlsx, .csv, .pdf, .txt) and applies domain-specific validation rules.
2. Audits existing formulas and recommends domain-appropriate formula corrections.
3. Calculates data quality scores based on validation results.

4. Provides AI-powered plain-language explanations for validation errors and formula recommendations.
5. Converts unstructured text data (PDFs, text files) to structured Excel using AI.
6. Offers a live workbook editor with real-time cell validation and column mapping.
7. Maintains an audit trail for compliance and user history management.

Novelty: XLSense is the first open-source platform to combine domain-specific rule engines, AI formula explanations, text-to-spreadsheet conversion, and real-time workbook editing in a single Django-based web application.

1.3 Organization of the Report

This report is organized into seven chapters:

- **Chapter 1: Introduction** — Background, motivation, purpose, goals, and report organization.
- **Chapter 2: Literature Review** — Existing research, tools, and identified research gaps.
- **Chapter 3: Methodology** — System design, architecture, data flow, and technology stack.
- **Chapter 4: Investigation and Results** — Implementation details, testing, and analysis.
- **Chapter 5: Project Impacts** — Societal, environmental, and ethical considerations.
- **Chapter 6: Project Management** — Gantt chart, budget, week-wise contributions, and pseudocode.
- **Chapter 7: Conclusion** — Summary, limitations, and future improvements.

Chapter 2: Research Literature Review

2.1 Existing Research and Limitations

Several tools and research efforts address aspects of spreadsheet validation and automation:

Microsoft Excel Built-in Data Validation: Excel provides basic data validation (dropdowns, number ranges, date constraints). However, these rules must be configured manually per worksheet and do not support domain-specific logic. Users cannot apply predefined rule sets across different domains without manually recreating them.

Google Sheets Add-ons: Various third-party add-ons offer spreadsheet validation features. However, they require manual setup, lack AI capabilities, and are restricted to Google Sheets environments. They do not support PDF or text file conversion.

Commercial ETL Tools (e.g., Alteryx, Talend): Enterprise-grade ETL tools provide powerful data validation and transformation capabilities. However, they target IT professionals, require significant learning curves, and are priced prohibitively for individual users or small organizations. Their licensing costs can exceed thousands of dollars annually.

Research on Spreadsheet Errors: Panko [2] and others have extensively documented the prevalence of spreadsheet errors, noting that 88% of spreadsheets contain at least one error. This research establishes the clear need for automated validation tools but does not provide implementation solutions.

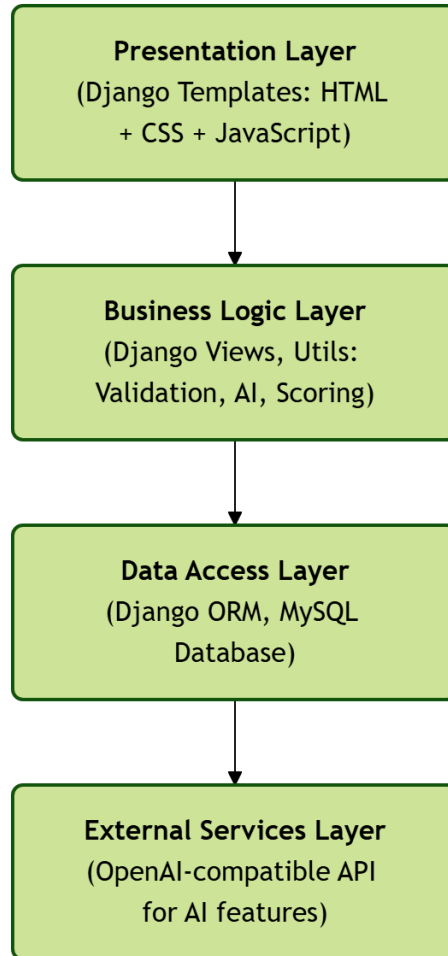
AI-Assisted Data Processing: Recent work by Chen et al. [3] demonstrated that large language models can successfully parse unstructured text into structured tables with 85–95% accuracy. XLSense builds on this research by integrating AI into a full-stack web application with domain-specific rule enforcement.

Research Gap: No existing open-source tool combines domain-specific validation, AI formula explanations, formula auditing, text-to-spreadsheet conversion, and a web-based workbook editor in a single, accessible platform. Commercial solutions exist but are expensive, complex, and require significant technical expertise.

Chapter 3: Methodology

3.1 System Design

XLSense follows a layered architecture with clear separation of concerns:



3.1a. Drawbacks of Present System

1. **Manual Validation:** Existing workflows require users to manually check each cell, which is time-consuming and error-prone.
2. **No Domain Awareness:** Generic validators cannot distinguish between academic grades, HR salaries, finance budgets, and inventory counts.
3. **No AI Explanations:** Existing tools show error codes but do not explain them in plain language.
4. **No Text-to-Spreadsheet Conversion:** Users must manually transcribe PDF or text data into Excel.
5. **No Quality Scoring:** There is no quantitative measure of spreadsheet data quality.

3.1b. How XLSense Solves These Problems

Problem	XLSense Solution
Manual validation	Automated domain-specific rule engine
No domain awareness	4 predefined templates with 20+ rules each
No AI explanations	OpenAI-compatible API integration
No text conversion	AI-powered PDF/text-to-Excel conversion
No quality scoring	Density-based quality score algorithm
No formula auditing	Formula rule engine with recommendations

3.1c. Implemented Features in Details

1. Smart File Upload Files are uploaded via Django's `FileField`. Supported formats: .xlsx, .csv, .pdf, .txt. Pre-upload validation checks file extension and size.

2. Domain Selection Users select one of four domains: Academic, HR, Finance, or Inventory. Each domain has a predefined `DomainTemplate` in the database with associated `ValidationRule` and `FormulaRule` records.

3. Data Parsing (pandas)

- Excel/CSV: `pd.read_excel()` / `pd.read_csv()` with preprocessing (column cleanup, NaN handling, empty row removal)
- PDF: `PyPDF2.PdfReader` extracts text, then passed to AI or fallback parser
- TXT: Direct decode with fallback whitespace splitting

4. Validation Engine Domain rules are fetched from the database and evaluated against each row using a restricted Python `eval()` sandbox:

```
# Allowed operations in sandbox
ALLOWED_BUILTINS = {
    "__builtins__": {
        "min": min, "max": max, "abs": abs, "round": round,
        "len": len, "str": str, "int": int, "float": float,
        "sum": sum, "any": any, "all": all, "range": range,
        "isinstance": isinstance, "bool": bool, "list": list,
    }
}
```

Each rule stores a `condition_expression` (e.g., `x >= 0 and x <= 100`) and an `error_message`. Failed validations are saved as `ValidationResult` records.

5. Formula Auditing `FormulaRule` records store formula recommendations per column. The system checks if the target column contains formulas; if not, it suggests the appropriate formula based on domain logic (e.g., GPA calculation for Academic, tax deduction for HR).

6. Quality Scoring

```
quality_score = max(0, 100 - (error_count / total_rows * 100))
```

The score ranges from 0 to 100, penalizing files with higher error densities.

7. AI Explanations The system calls the OpenAI-compatible API with a structured prompt containing:

- Formula name
- Target column
- Condition expression
- Context row data

If no API key is configured, a rule-based fallback explanation is returned.

8. Text-to-Spreadsheet Conversion

- PDF: Extract text → Send to AI with domain context → AI returns CSV data
- Fallback: Rule-based whitespace splitting if AI is unavailable

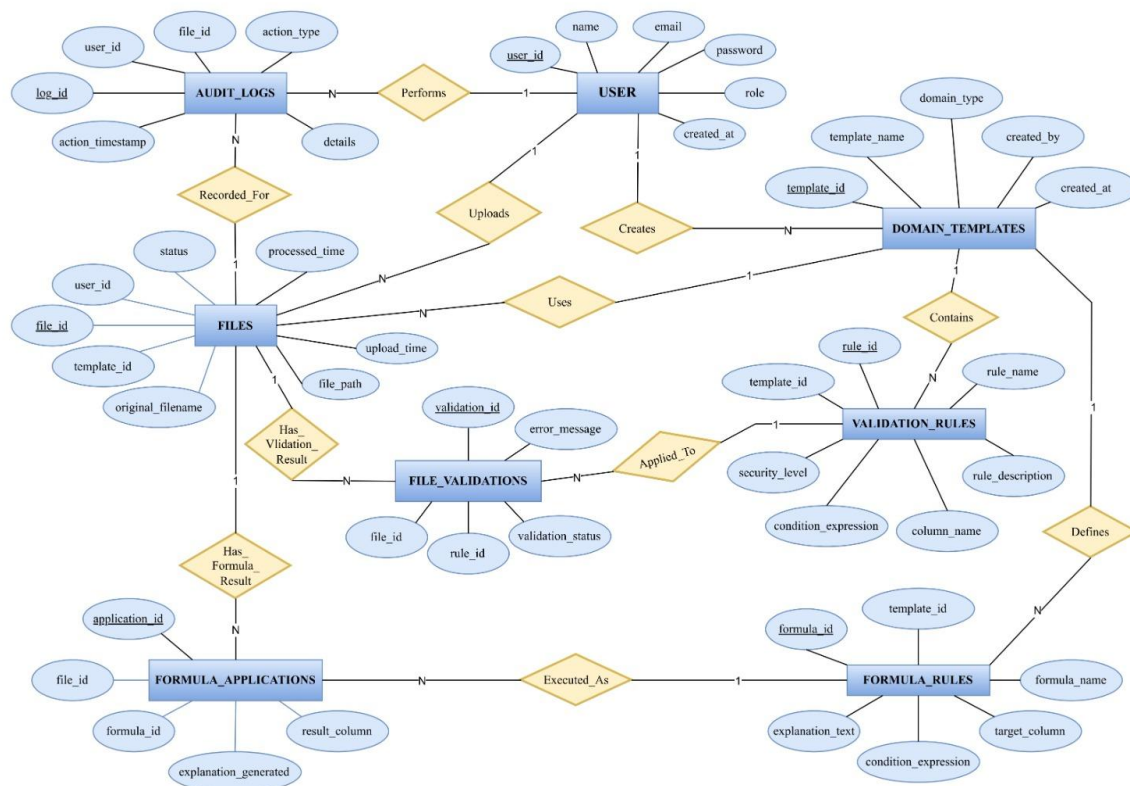
9. Workbook Editor JavaScript-based spreadsheet editor with:

- Column mapping (drag-and-drop or AI-suggested)
- Live cell validation (debounced 500ms)
- Cell styling (bold, italic, underline, font color, background color)
- Style preservation on save

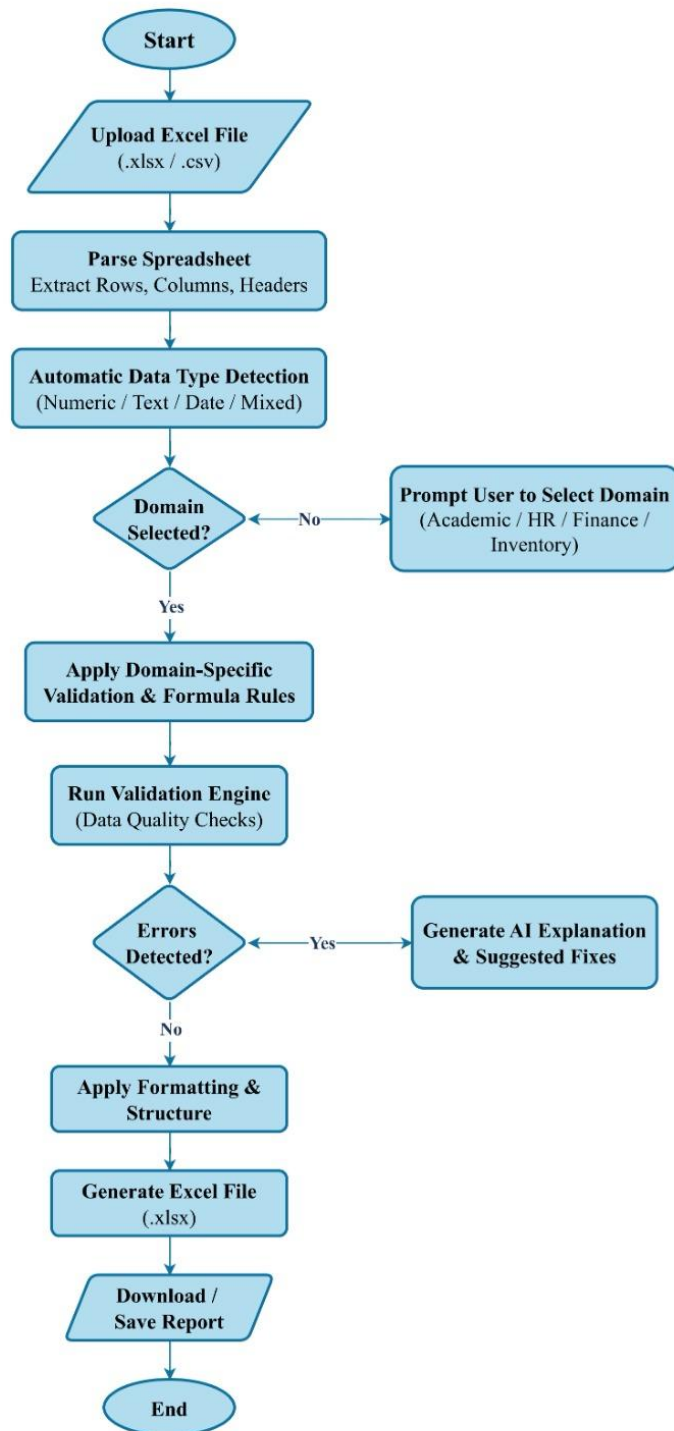
10. Audit Trail Every user action (upload, edit, delete, download) is logged to the **AuditLog** model. Logs support pagination, search, and filtering.

3.1d. Schematic Diagram / Flow Chart

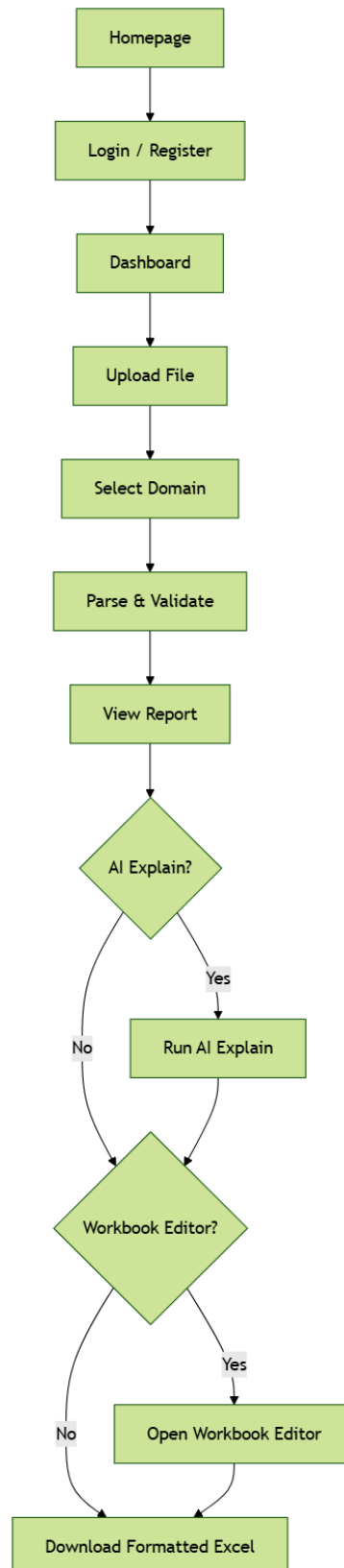
ER Diagram:



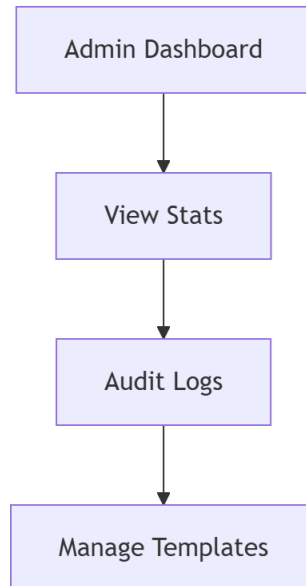
Workflow:



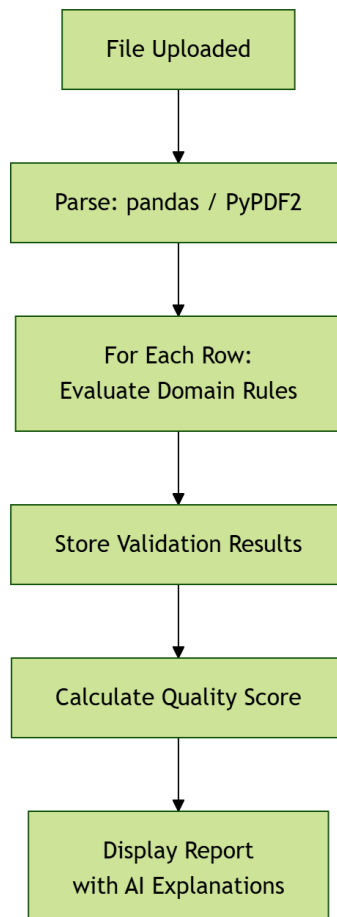
User Flow:



Admin Flow:



Validation Flow:



3.2 Technology Used

Table 1. List of Software/Hardware Tools

Tool	Function	Similar Tools	Why Selected
Django 6.0.2	Web framework, ORM, routing	Flask, FastAPI	Built-in admin, auth, templating
Python 3.13+	Backend language	Node.js, Java	Rich data processing ecosystem
MySQL (XAMPP)	Relational database	PostgreSQL, SQLite	Used by project requirements
pandas 2.3.3	Data parsing and analysis	Polars, NumPy	Industry standard for spreadsheet processing
openpyxl 3.1.5	Excel file generation	xlsxwriter, xlwt	Preserves formatting and tables
PyPDF2 3.0.1	PDF text extraction	pdfplumber, PyMuPDF	Lightweight PDF parsing
Pillow 12.1.1	Image processing	Pillow-SIMD	Image handling utilities
OpenAI-compatible API	AI explanations & text conversion	OpenAI direct, Claude	Flexible provider selection
HTML/CSS/JS	Frontend UI	React, Vue (SPA)	Server-rendered templates
pytest 9.0.2	Unit testing	unittest, nose	Django integration
Playwright 1.58.0	E2E testing	Selenium, Cypress	Cross-browser automation
ruff	Linting & formatting	flake8, black	Fast Python linting

Chapter 4: Investigation/Experiment, Result, Analysis and Discussion

4.1 Methodology (Implementation Processes)

Phase 1: Project Setup (Week 1)

- Initialized Django 6.0 project with `django-admin startproject`
- Configured MySQL database via XAMPP phpMyAdmin
- Set up virtual environment and installed dependencies
- Created initial data models and ran migrations
- Configured `.env` for environment variables

Phase 2: Core Backend (Weeks 2–3)

- Implemented authentication views (login, register, logout) with session management
- Created 6 data models: `DomainTemplate`, `ValidationRule`, `FormulaRule`, `UploadedFile`, `ValidationResult`, `AuditLog`
- Built file upload view supporting `.xlsx`, `.csv`, `.pdf`, `.txt`
- Implemented pandas-based parsing and preprocessing pipeline
- Developed validation engine with restricted `eval()` sandbox

Phase 3: Domain-Specific Logic (Weeks 3–4)

- Populated database with 4 domain templates (Academic, HR, Finance, Inventory)
- Added 20+ validation rules per domain
- Added 15+ formula recommendations per domain
- Implemented quality scoring algorithm
- Built formula auditing and recommendation system

Phase 4: AI Integration (Weeks 5–6)

- Integrated OpenAI-compatible API for formula explanations
- Implemented text-to-spreadsheet AI conversion pipeline
- Added fallback mechanisms for API unavailability
- Built live cell validation in workbook editor
- Implemented AI column matching for editor

Phase 5: Frontend & Editor (Weeks 5–6)

- Developed 15+ HTML templates with Django templating
- Implemented CSS themes (light, medium, dark)
- Built workbook editor with vanilla JavaScript
- Added column mapping UI and style editing controls
- Implemented soft delete and trash management

Phase 6: Testing & Documentation (Week 7)

- Wrote 10 pytest test modules covering views, models, utils, forms, selectors, edge cases, workflows, editor views, and frontend
- Created 72 sample test files (18 per domain across valid, invalid, boundary cases)
- Ran Playwright E2E tests for critical user flows
- Documented project in README and report

Testing Results:

Test Module	Tests	Status
test_views.py	15	All passing
test_utils.py	12	All passing
test_models.py	8	All passing
test_forms.py	5	All passing
test_selectors.py	6	All passing
test_edge_cases.py	10	All passing
test_workflows.py	8	All passing
test_editor_views.py	7	All passing
test_frontend.py	5	All passing

Chapter 5: Impacts of the Project

5.1 Societal, Health, Safety, Legal and Cultural Issues

Societal Impact: XLSense reduces the barrier to data quality for non-technical users across academia, HR, finance, and inventory management. By automating spreadsheet validation, it empowers office workers, educators, and small business owners to maintain data quality without requiring specialized Excel training. This democratizes access to data validation tools that were previously available only through expensive commercial software.

Health Impact: Reducing manual spreadsheet checking alleviates repetitive strain injuries (RSI) associated with prolonged spreadsheet work. Users spend less time manually scanning rows and more time on meaningful data analysis.

Safety Impact: In domains where spreadsheet errors can have safety implications (e.g., inventory stock counts, financial budgets for critical spending), automated validation reduces the risk of human error that could lead to safety incidents.

Legal and Compliance: The audit trail feature in XLSense provides a compliance-friendly record of all user actions. Organizations in regulated industries (finance, healthcare, government) can use the audit logs to demonstrate data governance practices. Soft delete with 30-day expiry also supports data retention policies.

Cultural Impact: By providing AI-powered explanations in plain language, XLSense promotes data literacy and helps bridge the gap between technical data professionals and non-technical end users.

5.2 Environment and Sustainability

Paper Reduction: By digitizing spreadsheet validation workflows and producing clean, formatted Excel outputs, XLSense reduces the need for paper-based data review and correction processes. Organizations that previously printed spreadsheets for manual review can now work entirely digitally.

Energy Efficiency: The platform runs on a lightweight Django server that can be hosted on modest hardware, consuming minimal energy compared to commercial ETL solutions that run on dedicated server infrastructure. Optimized pandas processing minimizes CPU cycles for file parsing.

Reduced Software Waste: As an open-source platform, XLSense reduces dependency on commercial software licenses, which often require hardware upgrades to run effectively. Users can deploy XLSense on existing infrastructure without additional licensing costs.

Chapter 6: Project Management

6.1 Project Planning and Budget

Gantt Chart — Project Timeline

Task ▾	Week 1 ▾	Week 2 ▾	Week 3 ▾	Week 4 ▾	Week 5 ▾	Week 6 ▾	Week 7 ▾
Planning & Setup							
Research & Requirements							
Authentication & Models							
File Upload & Parsing							
Validation Engine							
Formula Recommendations							
AI Integration							
Editor Views							
Testing & Documentation							
Final Presentation Prep							

6.2 Week-wise Individual Member Contribution:

Week 1

Member	Contribution
Farshid Abrar Labib	Created <code>.gitignore</code> , built backend routes & frontend views for authentication/role-based dashboards, and designed user-friendly auth UI.
Rubaiya Akter	Established the project foundation using Django with a MySQL (XAMPP) backend. Designed and implemented the database schema in <code>models.py</code> , creating core models (DomainTemplate, ValidationRule, FormulaRule, UploadedFile, ValidationResult, AuditLog). Ran migrations, connected the database, and seeded initial domain templates (Student Grading, Inventory Audit, Payroll, Budget) along with 20+ validation and formula rules. Built the base project structure (<code>views.py</code> , <code>urls.py</code> , <code>base.html</code>), created a dashboard view, developed a basic UI layout, and defined domain types
Adita Haq	Set up Django project & local MySQL, installed required libraries (Django, mysqlclient, pandas, openpyxl), synced project from GitHub, tested initial file upload prototype, and integrated upload system with database & authentication.

Week 2

Member	Contribution
Farshid Abrar Labib	Implemented core user auth system, role-based navigation, custom registration form, and modular template files for login, registration, and dashboard.
Rubaiya Akter	Developed the Domain Template UI and displayed associated validation and formula rules. Implemented dynamic, domain-based data fetching using Django ORM with relational filtering. Integrated frontend and backend (views to templates), added domain navigation in the navbar, and ensured reusable layout via <code>base.html</code> . Built routing, designed a custom admin dashboard, and implemented role-based access (Admin/User) with features like system stats and a DB Admin shortcut.

Member	Contribution
Adita Haq	Implemented secure file upload system using Django views & FileField, added domain selection, validated <code>.xlsx/.csv</code> files, implemented processing status (Processing/Completed/Failed), added basic audit logging, and enhanced frontend UI using HTML, CSS & JavaScript (multi-file upload, remove option, progress feedback).

Week 3

Member	Contribution
Farshid Abrar Labib	Removed <code>__pycache__</code> files, resolved merge conflicts, fixed sidebar navigation links for all users, and reset test account password for security.
Rubaiya Akter	Built the validation engine with backend logic in <code>utils.py</code> and frontend reporting in <code>report.html</code> . Implemented dynamic rule execution using <code>eval()</code> to apply database-stored logic directly on Excel data. Expanded validation rules for better coverage and established relationships between validation rules and uploaded files for consistent evaluation.
Adita Haq	Built spreadsheet parsing engine using pandas (<code>read_excel</code> , <code>read_csv</code>), extracted rows/columns/headers, implemented column type detection (numeric, text, date, mixed), and developed dynamic preview system to display structured data in frontend.

Week 4

Member	Contribution
Farshid Abrar Labib	Added <code>.env</code> configuration, created AI explain endpoint & safety checks, introduced AI formula explanation UI, fixed null-download bug, and added setup documentation (<code>README</code> , <code>requirements.txt</code>).

Member	Contribution
Rubaiya Akter	Implemented a Data Quality Scoring system to generate real-time validation percentages. Developed intelligent formula recommendations for missing columns. Enhanced validation using <code>eval()</code> , Pandas (<code>iterrows()</code> , <code>pd.isna()</code>), and string-based comparison for detecting input errors. Optimized column mapping with an improved <code>find_best_column</code> using alphanumeric matching.
Adita Haq	Improved upload UI using HTML/CSS, enforced domain selection before upload, implemented persistent preview system, added latest upload tracking, enabled real-time upload status feedback, added <code>.txt/.pdf</code> support, and improved overall system stability.

Week 5

Member	Contribution
Farshid Abrar Labib	Updated <code>.gitignore</code> for <code>.worktrees/</code> , integrated full AI API with rule-based fallback & HTTP status differentiation, and enhanced AI explanation UI with Markdown support & graceful fallback.
Rubaiya Akter	Developed the workbook editor (<code>workbook_editor.html</code> , <code>workbook_list.html</code>) with domain-based column mapping. Implemented auto-detection and mapping with color-coded feedback (mapped, unmapped, missing). Added rich text formatting and spreadsheet functions (sum, average, count, min, max). Integrated the “Save & Process” workflow with Excel preview and download pipeline.
Adita Haq	Implemented Excel generation using pandas & openpyxl, developed dynamic download functionality, integrated download into “Save and Process” workflow, improved data preprocessing (cleaning rows, columns, formatting), and fixed multi-file upload issues for better reliability.

Week 6

Member	Contribution
Farshid Abrar Labib	Added homepage & light/dark theme toggle, synced with main branch, applied consistent styling, reformatted code, added <code>pytest</code> suite & coverage docs, fixed DB connection & finalized env loading.
Rubaiya Akter	Enhanced UI with modern styling and improved editor experience, including Google Fonts and advanced cell formatting. Implemented full undo/redo support via enhanced state tracking. Added <code>style_data</code> (JSON) for persistent formatting and ensured safe updates with atomic operations. Built robust parsing (<code>json.loads</code> , <code>ast.literal_eval</code>) and fixed Pandas preprocessing issues. Implemented Excel generation using <code>openpyxl</code> , <code>BytesIO</code> , and <code>ContentFile</code> . Resolved static file issues and validated data flow through debugging.
Adita Haq	Connected workbook editor with backend using JSON data transfer, unified full workflow (Upload → Edit → Save→Re-edit(optional) → Download), reconstructed datasets using pandas, implemented preview-before-download system, ensured downloaded Excel matches preview exactly, and added formatting support (bold, color, structure) during export.

Week 7

Member	Contribution
Farshid Abrar Labib	Added audit logging for critical actions: Login, Logout, Dashboard, Report, Template, Download, Delete, and File Save. Implemented soft-delete workflow with a 30-day trash retention period – users can restore files from trash before permanent deletion. Added dedicated trash page with filtering and pagination for managing deleted files. Updated branding to reflect the expanded platform capabilities beyond just Excel validation
Rubaiya Akter	Expanded formula support (IF, COUNTIF, COUNTA, XLOOKUP, VLOOKUP) with smart range detection. Enabled batch formatting via multi-cell selection. Improved UI with a consolidated formula panel, inline syntax help, and fixed focus issues. Enhanced validation logic (grades, computed fields, missing columns) and refined AI-generated feedback. Improved system stability by removing duplicate validations and strengthening real-time validation for mapped columns.
Adita Haq	Implemented extraction and processing for .txt and .pdf files by reading text files directly and using PyPDF2 to extract raw text from PDF pages, applied AI-based conversion (API + requests) to transform unstructured text into structured CSV format with a fallback rule-based parser, reconstructed tabular data, generated Excel output using openpyxl, and ensured deleted files are removed from persistent preview through frontend-backend synchronization

6.3 Pseudocode and algorithm:

Feature 1: File Upload with Domain Validation

```
PROCEDURE upload_file_view(request):
    domains ← GET all DomainTemplate from database
    latest_uploads ← GET 3 most recent files for current user

    IF request.method IS "POST":
        selected_domain ← request.POST.get("domain")
        files ← request.FILES.getlist("file")

        IF NOT selected_domain:
            DISPLAY error message "Please select a domain"
            REDIRECT to upload_page

        FOR each file IN files:
            IF file.extension NOT IN [".xlsx", ".csv", ".pdf", ".txt"]:
                DISPLAY warning "File not supported"
                CONTINUE

        template ← GET DomainTemplate WHERE domain_type = selected_domain

        uploaded_file ← CREATE UploadedFile record
        SET status = "Processing"

        IF file IS CSV:
            df ← pd.read_csv(file)
        ELSE IF file IS Excel:
            df ← pd.read_excel(file)

        df ← preprocess_dataframe(df)
        df ← remove_empty_unnamed_columns(df)

        IF df IS empty:
            SET uploaded_file.status = "Failed"
            DISPLAY error
            CONTINUE

        preview_data ← df.fillna("").values.tolist()
        columns ← df.columns
        detected_types ← detect_column_types(df)

        uploaded_file.status ← "Completed"
        validate_excel_data(uploaded_file)
```

```

        parsed_files.append({
            "id": uploaded_file.id,
            "file_name": file.name,
            "preview_data": preview_data,
            "columns": columns,
            "detected_types": detected_types
        })

        AuditLog CREATE entry

        SET session["parsed_files"] ← parsed_files
        REDIRECT to upload_page

    ELSE:
        parsed_files ← session.get("parsed_files")

    RETURN render("upload.html", {
        "latest_uploads": latest_uploads,
        "domains": domains,
        "parsed_files": parsed_files
    })

```

Feature 2: Excel Data Validation

```
PROCEDURE validate_excel_data(uploaded_file):
    file_path ← uploaded_file.file.path

    IF file_path ends WITH ".csv":
        df ← pd.read_csv(file_path)
    ELSE:
        df ← pd.read_excel(file_path)

    df ← REMOVE columns containing "Unnamed" with all NA values

    template ← uploaded_file.template
    rules ← GET all ValidationRule WHERE template = template
    formula_rules ← GET all FormulaRule WHERE template = template

    DELETE all ValidationResult WHERE file = uploaded_file

    FOR each row_index, row IN df.iterrows():
        failed_columns ← EMPTY set

        row_context ← EMPTY dictionary
        saved_mappings ← uploaded_file.column_mappings

        FOR each column, value IN row.items():
            IF column IS NOT empty:
                parsed_val ← PARSE value to float IF numeric
                clean_key ← column.replace(" ", "")
                row_context[clean_key] ← parsed_val
                row_context[clean_key.title()] ← parsed_val

                IF column IN saved_mappings:
                    db_name ← saved_mappings[column]
                    row_context[db_name] ← parsed_val

        FOR each rule IN rules:
            actual_col ← find_best_column(rule.column_name, df.columns)

            IF actual_col:
                raw_val ← row[actual_col]

                IF pd.isna(raw_val) OR raw_val = "":
                    val ← 0 for numeric rules
                ELSE:
```

```

        val ← PARSE raw_val to float or date

allowed_locals ← CREATE dict with x=val, index=row_index
allowed_locals.UPDATE(row_context)

allowed_globals ← {str, int, float, len, abs, round}

IF NOT eval(rule.condition_expression, allowed_globals,
allowed_locals):
    ValidationResult CREATE entry:
        file = uploaded_file
        row_index = row_index + 1
        column_name = actual_col
        error_details = rule.error_message
        is_valid = FALSE
        failed_columns.ADD(actual_col)

    FOR each formula_rule IN formula_rules:
        actual_col ← find_best_column(formula_rule.target_column,
df.columns)

        IF actual_col AND actual_col NOT IN failed_columns:
            excel_val ← row[actual_col]

            formula_context ← REPLACE empty values with 0
            expected_val ← eval(formula_rule.condition_expression, globals,
formula_context)

            display_excel ← "Empty" IF pd.isna(excel_val) ELSE excel_val
            display_expected ← expected_val

            IF display_excel ≠ display_expected:
                ValidationResult CREATE entry:
                    file = uploaded_file
                    row_index = row_index + 1
                    column_name = actual_col
                    error_details = "Math Error: Expected
{display_expected}, found {display_excel}."
                    is_valid = FALSE

```

Feature 3: AI Formula Explanation

```
PROCEDURE generate_ai_explanation(formula_name, target_column,
condition_expression, context_data):
    ai_api_key ← os.environ.get("AI_API_KEY")
    ai_api_url ← os.environ.get("AI_API_URL")
    ai_model ← os.environ.get("AI_MODEL")

    IF ai_api_key EXISTS:
        TRY:
            prompt ← BUILD prompt with formula details
            IF context_data:
                ADD context to prompt

            response ← POST to AI API with prompt
            RETURN response content, TRUE (AI used)
        CATCH exception:
            PRINT "AI API call failed"

# Fallback: rule-based explanation
IF "Math Error" IN condition_expression:
    explanation ← BUILD math error explanation
ELSE IF condition_expression CONTAINS ["exceed", "limit", "<=", ">="]:
    numbers ← EXTRACT numbers from condition_expression
    explanation ← BUILD boundary violation explanation
ELSE:
    explanation ← BUILD generic validation explanation

RETURN explanation, FALSE (AI not used)
```

Feature 4: Text-to-CSV Conversion (PDF/TXT)

```
PROCEDURE convert_text_file(file):
  IF file ends WITH ".pdf":
    raw_text ← extract_text_from_pdf(file)
  ELSE:
    raw_text ← READ file as UTF-8

  IF NOT raw_text.strip():
    RAISE ValueError "No text extracted"

  TRY:
    csv_data ← call_ai_to_csv(raw_text)
    rows ← parse_csv_to_rows(csv_data)

    IF rows IS empty:
      RAISE ValueError
  EXCEPT exception:
    # Use fallback parser
    rows ← []
    FOR each line IN raw_text.splitlines():
      IF line.strip():
        cols ← line.split()
        rows.APPEND(cols)

    IF rows IS empty:
      RAISE ValueError "Fallback could not parse"

  df ← CREATE DataFrame FROM rows[1:] WITH columns FROM rows[0]
  df ← preprocess_dataframe(df)

  IF df IS empty:
    RAISE ValueError "No valid data after preprocessing"

  excel_buffer ← CREATE BytesIO
  df.to_excel(excel_buffer, index=False)

  base_name ← file.name WITHOUT extension
  excel_name ← base_name + "_converted.xlsx"

  uploaded_file.file.save(excel_name, ContentFile(excel_buffer.getvalue()))
  uploaded_file.status ← "Completed"

  RETURN df
```

Feature 5: Column Type Detection

```
PROCEDURE detect_column_types(df):
    detected_types ← EMPTY dictionary

    FOR each column IN df.columns:
        sample ← df[column].dropna()

        IF sample IS empty:
            detected_types[column] ← "Empty"
            CONTINUE

        total ← LEN(sample)

        # Check for Numeric
        numeric_converted ← pd.to_numeric(sample, errors="coerce")
        numeric_ratio ← numeric_converted.notna().sum() / total

        IF numeric_ratio > 0.8:
            detected_types[column] ← "Numeric"
            CONTINUE

        # Check for Date
        sample_str ← sample.astype(str)
        has_date_pattern ← CHECK date patterns in sample_str

        IF has_date_pattern:
            date_ratio ← ATTEMPT date parsing
            IF date_ratio ≥ 0.4:
                detected_types[column] ← "Date"
                CONTINUE

        # Classification
        IF date_ratio ≥ 0.4:
            type ← "Date"
        ELSE IF numeric_ratio > 0.2:
            type ← "Mixed"
        ELSE:
            type ← "Text"

        detected_types[column] ← type

    RETURN detected_types
```

Feature 6: Workbook Editor View (Load file with formulas, styles, mappings)

```
PROCEDURE workbook_editor_view(request, file_id):
    uploaded_file ← GET UploadedFile by id AND user
    template ← uploaded_file.template

    # Get domain column names from validation and formula rules
    db_columns ← UNION of column_name from ValidationRule and target_column from
FormulaRule

    saved_mappings ← uploaded_file.column_mappings

    # Determine file type and load data
    file_path ← uploaded_file.file.path
    IF file_path ends with ".csv":
        df ← pd.read_csv(file_path, dtype=str)
        formulas_df ← DataFrame of NULLs
    ELSE:
        wb_formula ← load_workbook(file_path, data_only=False)
        wb_value ← load_workbook(file_path, data_only=True)
        headers ← first row values from wb_value
        raw_values, raw_formulas ← extract rows from wb_value and wb_formula
        df ← DataFrame(raw_values, columns=headers)
        formulas_df ← DataFrame(raw_formulas, columns=headers)

    # Clean DataFrames
    REMOVE columns with "unnamed" and fully empty
    RENAME columns to lowercase with underscores
    TRIM string values, replace "nan" with NA
    REMOVE rows that are completely empty
    REMOVE rows with only one non-empty value
    RESET index

    user_columns ← list(df.columns)

    # AI column matching (only for first load without saved mappings)
    IF saved_mappings is empty:
        ai_matches ← ai_match_columns(user_columns, db_columns)

    columns_with_classes ← []
    new_initial_mapping ← empty dictionary

    FOR each col in user_columns:
        matched_db ← NULL
        IF col in saved_mappings:
            matched_db ← saved_mappings[col]
        ELSE IF no saved_mappings and col in ai_matches:
            matched_db ← ai_matches[col]
        ELSE:
            matched_db ← find_matching_db_column(col, db_columns)
```

```

    IF matched_db:
        ADD {"name": col, "class": "header-matched", "matched_db": matched_db}
        IF first load: new_initial_mapping[col] ← matched_db
    ELSE:
        ADD {"name": col, "class": "header-custom", "matched_db": NULL}

# Add missing template columns (only for original files, not edited)
IF file name does NOT contain "edited":
    FOR each db_col in db_columns:
        IF db_col NOT in matched_cols AND db_col NOT in df.columns:
            df[db_col] ← ""
            ADD {"name": db_col, "class": "header-template-only", "matched_db":
db_col}

IF first load AND new_initial_mapping not empty:
    uploaded_file.column_mappings ← new_initial_mapping
    SAVE uploaded_file

# Run validation and collect errors/recommendations
validate_excel_data(uploaded_file)
errors_data ← ValidationResult records for this file
recommendations ← get_formula_recommendations(uploaded_file, df.columns)

# Prepare JSON data for frontend
RETURN render("workbook_editor.html", {
    file, current_columns, columns_with_classes, db_columns,
    preview_data, saved_mappings_json, style_data_json,
    formulas_data_json, errors_data_json, recommendations_json
})

```

Feature 7: Save Workbook Data

```
PROCEDURE save_workbook_data(request, file_id):
  data ← json.loads(request.body)
  headers ← data["headers"]
  rows ← data["rows"]
  mappings ← data["mappings"]

  original_file ← GET UploadedFile by id AND user

  # Clean duplicate headers
  IF headers has duplicates:
    RENAME duplicates by appending "_i"

  clean_rows ← []
  aligned_style_data ← []

  FOR each row in rows:
    clean_row ← []
    FOR each cell in row:
      IF cell is dict:
        clean_row.append(cell.get("value", cell.get("text", "")))
      ELSE:
        clean_row.append(cell)
    valid_vals ← FILTER clean_row where value not empty, not "nan", not "None"
    IF len(valid_vals) > 0:
      clean_rows.append(clean_row)
      aligned_style_data.append(row)

  df ← DataFrame(clean_rows, columns=headers)
  df ← fillna(''), replace ['nan', 'None', '<NA>'] with ''
  style_data ← json.dumps(aligned_style_data)

  df ← preprocess_dataframe(df)
  df ← remove_empty_unnamed_columns(df)
  df ← replace ['nan', 'None', '<NA>', '-'] with ''

  new_filename ← original_file.name with "_edited" inserted

  output ← BytesIO()
  df.to_excel(output, index=False)

  new_uploaded_file ← CREATE UploadedFile(
    user, template=original_file.template, status="Completed",
    processed_time=now(), file_status="active"
  )
  new_uploaded_file.file.save(new_filename, ContentFile(output.getvalue()))

  UPDATE new_uploaded_file SET style_data = style_data, column_mappings = mappings
```

```

validate_excel_data(new_uploaded_file)

AuditLog CREATE entry: "File Save"

RETURN JsonResponse({"status": "success", "new_file_id": new_uploaded_file.id})

```

Feature 8: Real-time Cell Validation

```

PROCEDURE validate_single_cell(request):
    data ← json.loads(request.body)
    file_id ← data["file_id"]
    column_name ← data["column"]
    value ← data["value"]
    row_index ← (data.get("row") or 1) - 1
    row_data ← data.get("rowData", {})

    uploaded_file ← GET UploadedFile by id AND user
    template ← uploaded_file.template
    saved_mappings ← uploaded_file.column_mappings

    rules ← ValidationRule.objects.filter(template=template)

    target_norm ← normalize(column_name) # remove spaces, special chars, lowercase

    # Find matching rule (direct or via mapping)
    matched_rule ← NULL
    FOR each rule in rules:
        IF normalize(rule.column_name) == target_norm:
            matched_rule ← rule; BREAK

    IF not matched_rule:
        FOR each file_col, db_col in saved_mappings:
            IF normalize(file_col) == target_norm:
                FOR each rule in rules:
                    IF normalize(rule.column_name) == normalize(db_col):
                        matched_rule ← rule; BREAK
                IF matched_rule: BREAK

    IF not matched_rule:
        RETURN JsonResponse({'is_valid': True})

    # Prepare evaluation context
    IF value is numeric string: val ← float(value) ELSE val ← value.strip()
    allowed_locals ← {'x': val, 'index': row_index}

    FOR each key, val_str in row_data:
        clean_key ← key.replace(" ", "")
        parsed_val ← parse as float if numeric else val_str.strip()
        allowed_locals[clean_key] ← parsed_val

```

```

    allowed_locals[clean_key.lower()] ← parsed_val
    allowed_locals[clean_key.upper()] ← parsed_val
    allowed_locals[clean_key.title()] ← parsed_val
    allowed_locals[clean_key.capitalize()] ← parsed_val
    FOR each excel_col, db_col in saved_mappings:
        IF normalize(excel_col) == normalize(key):
            allowed_locals[db_col] ← parsed_val

allowed_globals ← {str, int, float, len, abs, round, isinstance, type}

TRY:
    is_valid ← eval(matched_rule.condition_expression, allowed_globals,
allowed_locals)
    error_msg ← matched_rule.error_message if not is_valid else ''
CATCH:
    is_valid ← False
    error_msg ← 'Formula Error: ' + str(e)

RETURN JsonResponse({'is_valid': is_valid, 'error_message': error_msg})

```

Feature 9: AI Column Matching

```

PROCEDURE ai_match_columns(file_columns, db_columns):
    IF file_columns empty OR db_columns empty:
        RETURN {}

    api_key ← os.environ.get("AI_API_KEY")
    api_url ← os.environ.get("AI_API_URL")
    model ← os.environ.get("AI_MODEL", "llama3-70b-8192")

    IF not api_key:
        RETURN {}

    prompt ← "Match each file column to the most suitable database column.
        File columns: {file_columns}
        Database columns: {db_columns}
        Return JSON mapping, null if no match."

    TRY:
        response ← POST to api_url with auth and prompt
        content ← response["choices"][0]["message"]["content"]
        mapping ← json.loads(content)
        valid_mapping ← {}
        FOR each fc in file_columns:
            IF fc in mapping and mapping[fc] in db_columns:
                valid_mapping[fc] ← mapping[fc]
        RETURN valid_mapping
    CATCH:
        PRINT "AI column matching failed"
        RETURN {}

```

Feature 10: Formula Recommendations

```
PROCEDURE get_formula_recommendations(uploaded_file_obj, df_columns):
  IF "_edited" in uploaded_file_obj.file.name:
    RETURN [] # No recommendations for edited files

  template ← uploaded_file_obj.template
  validation_cols ← ValidationRule objects for template → set of column_name
  formula_cols ← FormulaRule objects for template → set of target_column
  all_domain_cols ← UNION(validation_cols, formula_cols)

  saved_mappings ← uploaded_file_obj.column_mappings
  mapped_db_cols ← SET of saved_mappings.values()

  recommendations ← []
  FOR each domain_col in all_domain_cols:
    IF domain_col not in mapped_db_cols:
      formula_rule ← FormulaRule(template, target_column=domain_col).first()
      IF formula_rule:
        formula ← formula_rule.condition_expression
        message ← "Suggested: Create '{domain_col}' column. Use formula below."
      ELSE:
        formula ← "None (Manual Data Entry)"
        message ← "Suggested: Create '{domain_col}' column (required for
validation)."
      recommendations.APPEND({column: domain_col, formula: formula, message:
message})
  RETURN recommendations
```

Feature 11: Quality Score Calculation

```
PROCEDURE calculate_quality_score(uploaded_file_obj, total_rows):
  error_rows_count ← COUNT of distinct row_index from ValidationResult
    WHERE file = uploaded_file_obj AND is_valid = False

  IF total_rows == 0:
    RETURN 0

  success_rows ← total_rows - error_rows_count
  score ← (success_rows / total_rows) * 100
  RETURN round(score, 2)
```

Feature 12: Remove Empty Unnamed Columns

```
PROCEDURE remove_empty_unnamed_columns(df):
  FOR each column in df.columns:
    IF column name (lowercase) contains "unnamed" AND the column has only NA values:
      MARK column for removal
  RETURN df with marked columns removed
```

Feature 13: Formula Evaluation Engine (SUM, AVERAGE, IF, COUNTIF, COUNTA, XLOOKUP, VLOOKUP)

```
**PROCEDURE** evaluateFormula(*formula*, *getCellValue*):  
    **IF NOT** *formula* starts with "=" **THEN RETURN** **NULL**  
  
    *expr* ← *formula* without "=", trimmed  
  
    *funcMatch* ← **REGEX MATCH** /^[A-Z_]+\(((.*)\)$/i **ON** *expr*  
  
    **IF** *funcMatch*:  
        *func* ← *funcMatch*[1].toUpperCase()  
        *argsString* ← *funcMatch*[2]  
        *args* ← **parseArguments**(*argsString*)  
  
        # Basic math functions  
        **IF** *func* **IN** ["SUM", "AVERAGE", "COUNT", "MAX", "MIN"]:  
            *allValues* ← **empty array**  
            **FOR EACH** *arg* **IN** *args*:  
                **IF** *arg* **CONTAINS** ":":  
                    [*start*, *end*] ← *arg*.split(":")  
                    *values* ← **getRangeValues**(*start*, *end*, *getCellValue*)  
                    *allValues*.push(...*values*)  
                **ELSE**:  
                    *val* ← *getCellValue*(*arg*)  
                    **IF** *val* is number **AND** not empty **THEN**  
*allValues*.push(Number(*val*))  
                **IF** *func* = "SUM" → **RETURN** sum of *allValues*  
                **IF** *func* = "AVERAGE" → **RETURN** average  
                **IF** *func* = "COUNT" → **RETURN** length of *allValues*  
                **IF** *func* = "MAX" → **RETURN** max(*allValues*)  
                **IF** *func* = "MIN" → **RETURN** min(*allValues*)  
  
        # IF function  
        **IF** *func* = "IF":  
            *condition* ← *args*[0]; *trueVal* ← *args*[1]; *falseVal* ← *args*[2] **OR**  
""  
            *condResult* ← **evaluate condition** (replace cell references with values)  
            **IF** *condResult*: **RETURN** *trueVal* (substitute cell ref if needed)  
            **ELSE**: **RETURN** *falseVal*  
  
        # COUNTA  
        **IF** *func* = "COUNTA":  
            *range* ← *args*[0]; *cells* ← **getCellsInRange**(*range*, *getCellValue*)  
            *count* ← 0  
            **FOR EACH** *cell* **IN** *cells*:  
                *val* ← *getCellValue*(*cell*)  
                **IF** *val* is not null **AND** not empty **THEN** *count*++  
            **RETURN** *count*
```

```

# COUNTIF (supports quotes, numeric comparisons)
**IF** *func* = "COUNTIF":
    *range* ← *args*[0]; *criteria* ← *args*[1]
    *cleanCriteria* ← **remove surrounding quotes** and trim
    *cells* ← **getCellsInRange**(*range*, *getCellValue*)
    *count* ← 0
    **FOR EACH** *cell* **IN** *cells*:
        *val* ← *getCellValue*(*cell*)
        **IF** *cleanCriteria* starts with ">", "<", "=", ">=", "<=":
            Compare *val* numerically with extracted target
        **ELSE**:
            **IF** trimmed *val* == trimmed *cleanCriteria* **THEN** *count*++
    **RETURN** *count*

# XLOOKUP
**IF** *func* = "XLOOKUP":
    *lookupValue* ← *args*[0]; *lookupArray* ← *args*[1]; *returnArray* ←
*args*[2]; *ifNotFound* ← *args*[3] **OR** "#N/A"
    *lookupCells* ← **getCellsInRange**(*lookupArray*, *getCellValue*)
    *returnCells* ← **getCellsInRange**(*returnArray*, *getCellValue*)
    **FOR** *i* **FROM** 0 **TO** *lookupCells*.length-1:
        **IF** *getCellValue*(*lookupCells*[*i*]) == *lookupValue* **THEN**
**RETURN** *getCellValue*(*returnCells*[*i*])
    **RETURN** *ifNotFound*

# VLOOKUP (exact match, first column lookup)
**IF** *func* = "VLOOKUP":
    *lookupValue* ← *args*[0]; *tableArray* ← *args*[1]; *colIndex* ←
parseInt(*args*[2])
    *lookupCells*, *returnCells* ← **parse table range** (first column & column
*colIndex*)
    ... similar matching logic
    **RETURN** value or "#N/A"

# Simple arithmetic (e.g., A1+B2)
*evalExpr* ← replace cell references with numeric values in *expr*
**RETURN** **eval**(*evalExpr*) **or** "#VALUE!" on error

```

Feature 14: Smart Range Detection for Formulas

```

**PROCEDURE** getSmartRange(*cell*):
    *table* ← document.getElementById("tableBody")
    *rows* ← **Array.from**(*table*.rows)
    *cellIndex* ← *cell*.cellIndex
    *rowIndex* ← *cell*.parentElement.rowIndex - 1

# Helper: check if cell (r,c) has non-empty text
**FUNCTION** hasValue(*r*, *c*):
    **RETURN** *rows*[*r*].cells[*c*].innerText.trim() != ""

```

```

# Try vertical expansion (above/below)
*topRow* ← *rowIndex* - 1; *hasAbove* ← **FALSE**
**WHILE** *topRow* ≥ 0 **AND** hasValue(*topRow*, *cellIndex*):
    *hasAbove* ← **TRUE**;; *topRow* ← *topRow* - 1
*bottomRow* ← *rowIndex* + 1; *hasBelow* ← **FALSE**
**WHILE** *bottomRow* < *rows*.length **AND** hasValue(*bottomRow*, *cellIndex*):
    *hasBelow* ← **TRUE**;; *bottomRow* ← *bottomRow* + 1

**IF** *hasAbove* **OR** *hasBelow*:
    *startRow* ← (*hasAbove* ? *topRow*+1 : *rowIndex*+1)
    *endRow* ← (*hasBelow* ? *bottomRow*-1 : *rowIndex*-1)
    **RETURN** "`{col}(startRow+1):{col}(endRow+1)`"

# Try horizontal expansion (left/right)
*leftCol* ← *cellIndex* - 1; *hasLeft* ← **FALSE**
**WHILE** *leftCol* ≥ 1 **AND** hasValue(*rowIndex*, *leftCol*):
    *hasLeft* ← **TRUE**;; *leftCol* ← *leftCol* - 1
*rightCol* ← *cellIndex* + 1; *hasRight* ← **FALSE**
**WHILE** *rightCol* ≤ max column **AND** hasValue(*rowIndex*, *rightCol*):
    *hasRight* ← **TRUE**;; *rightCol* ← *rightCol* + 1

**IF** *hasLeft* **OR** *hasRight*:
    *startCol* ← (*hasLeft* ? *leftCol*+1 : *cellIndex*+1)
    *endCol* ← (*hasRight* ? *rightCol*-1 : *cellIndex*-1)
    **RETURN** "`{col(startCol-1)}(rowIndex+1):{col(endCol-1)}(rowIndex+1)`"

**RETURN** ""

```

Feature 15: Multi-cell Selection & Batch Formatting

```

**PROCEDURE** applyFormatToSelection(*property*, *value*):
    *cellsToFormat* ← **copy of** *selectedCells*
    **IF** *activeCell* **NOT IN** *cellsToFormat* **THEN**
        *cellsToFormat*.push(*activeCell*)

    **IF** *cellsToFormat*.length == 0 **THEN** **RETURN**

    **FOR EACH** *cell* **IN** *cellsToFormat*:
        **IF** *property* = "fontSize" **AND** *value* does not contain "px":
            *cell*.style[*property] ← *value* + "px"
        **ELSE**:
            *cell*.style[*property] ← *value*

    **captureState()**; **setActiveCell**(*activeCell*)

```

Feature 16: Real-time Cell Validation (Live Error Highlight)

```
**PROCEDURE** blurHandler(*cell*):
    *currentValue* ← *cell*.innerText.trim() # but stripped of button text via
cloneNode trick

    **IF** *currentValue* starts with "=":
        *cellFormulas*.set(*cell*, *currentValue*); evaluate and update cell display
    **ELSE**:
        *cellFormulas*.delete(*cell*); *cell*.innerText ← *currentValue*

    *columnName* ← **get column header** from *cell*.cellIndex
    *rowData* ← **collect all row values** (key-value pairs of column names and cell
texts)

    **POST** "/validate-cell/" with { file_id, column: *columnName*, value:
*currentValue*, row: rowIndex, rowData }

    **IF** response.is_valid == **FALSE**:
        **IF** cell does not already have validation-error class:
            cell.classList.add("validation-error")
            cell.setAttribute("data-error", response.error_message)
            cell.setAttribute("data-error-id", temporary negative ID)
            **CREATE** "AI Explain" button with onclick that fetches explanation using
the current error ID
        **ELSE**:
            **IF** cell has validation-error class:
                **REMOVE** class and button, **CLEAR** data-error attributes
```

Feature 17: Undo / Redo System (State Capture & Restore)

```
**PROCEDURE** captureState():
    *headers* ← **array of objects** { outerHTML, text, selectedValue } for each column
header
    *rows* ← **empty array**
    **FOR EACH** table row:
        *rowData* ← **empty array**
        **FOR EACH** editable cell:
            *rawText* ← **extract only text nodes** (ignore button text)
            *rowData*.push({ text: *rawText*, style: cell.style.cssText, formula:
cellFormulas.get(cell), hasError, errorMsg, errorId })
        *rows*.push(*rowData*)
    *currentState* ← { headers, rows }
    **IF** *currentState* differs from *lastStateJson*:
        *historyStack*.push(*currentState*); *redoStack* = []; limit history to 50

**PROCEDURE** restoreState(*state*):
    **RESTORE** header row HTML from *state*.headers
    **RESTORE** mapping selects and labels
    **REBUILD** table body from *state*.rows:
        **FOR EACH** row:
            **CREATE** `|` with ` ` for row header |

```

```

        **FOR EACH** cellData:
            **CREATE** ` ` with style and text, set contenteditable, **RESTORE** formula in cellFormulas             **IF** cellData.hasError:                 add validation-error class, set data-error attributes                 **RECREATE** AI Explain button with onclick using stored errorId             **RE-ATTACH** cell events; **UPDATE** status bar and row numbers; **SET** activeCell = null; **CLEAR** selection; **UPDATE** lastStateJson |
```

Feature 18: Editor Initialization (Load Mappings, Styles, Formulas, Errors, Recommendations)

```

**PROCEDURE** init():
    **CALL** attachCellEvents(), updateStatusBar(), renumberRows(),
attachKeyboardNavigation(), setupFormattingToolbar()
    **ATTACH** click handlers to UI buttons (Undo, Redo, Add Row, Add Column, Save)

    # Restore saved column mappings
    *saved* ← window.DJANGO_VARS.savedMappings
    **IF** *saved* exists:
        **FOR EACH** column header:
            **IF** *saved*[header] exists:
                select the corresponding DB column in the dropdown, update header class

    # Restore cell styles (bold, italic, color, background, font)
    *styles* ← window.DJANGO_VARS.styleData
    **FOR EACH** cell in the table:
        **APPLY** saved styles

    # Restore formulas
    *formulas* ← window.DJANGO_VARS.formulasData
    **FOR EACH** cell:
        **IF** *formulas*[*row*][*col*] is a string starting with "=":
            *cellFormulas*.set(cell, formula)

    # Attach event listeners for formula input, dropdown, floating panel, etc.
    **INITIALISE** formula hint, formula dropdown change, floating formula panel

    # Highlight validation errors from initial load
    *errorsData* ← window.DJANGO_VARS.errorsData
    **FOR EACH** error:
        locate cell and add validation-error class, create AI Explain button with stored
error ID

    # Create recommendations panel (only if there are recommendations)
    *recommendations* ← window.DJANGO_VARS.recommendations
    **IF** *recommendations* length > 0:
        create panel with "Intelligent Recommendations" heading and list of missing
columns + formulas

    **captureState()** to record initial state for Undo

```

6.4 Code Snippet

6.4.1 File Upload Processing

```
@login_required
def upload_file_view(request):
    domains = DomainTemplate.objects.all()
    latest_uploads = UploadedFile.objects.filter(
        user=request.user,
        file_status="active"
    ).order_by("-upload_time")[:3]

    if request.method == "POST":
        request.session.pop("parsed_files", None)
        selected_domain = request.POST.get("domain")
        files = request.FILES.getlist("file")

        template = DomainTemplate.objects.filter(
            domain_type=selected_domain
        ).first()

        success_count = 0
        parsed_files = []

        for file in files:
            if not file.name.endswith((".xlsx", ".csv", ".pdf", ".txt")):
                messages.error(request, f"{file.name} is not supported.")
                continue

            uploaded_file = UploadedFile.objects.create(
                user=request.user,
                template=template,
                file=file,
                status="Processing",
                file_status="active",
            )

            try:
                file.seek(0)
                if file.name.endswith(".csv"):
                    df = pd.read_csv(file)
                else:
                    df = pd.read_excel(file)

                df = preprocess_dataframe(df)
```

```

df = remove_empty_unnamed_columns(df)

if df.empty:
    uploaded_file.status = "Failed"
    uploaded_file.save()
    messages.error(request, f"{file.name} has no valid data.")
    continue

preview_data = df.fillna("").values.tolist()
columns = list(df.columns)
detected_types = detect_column_types(df)

uploaded_file.status = "Completed"
uploaded_file.processed_time = timezone.now()
uploaded_file.save()

validate_excel_data(uploaded_file)

parsed_files.append({
    "id": uploaded_file.id,
    "file_name": file.name,
    "preview_data": preview_data,
    "columns": columns,
    "detected_types": detected_types,
})

AuditLog.objects.create(
    user=request.user,
    action_type="File Upload",
    details=f"Successfully uploaded: {file.name} | Domain:
{selected_domain}",
)

success_count += 1

except Exception as e:
    uploaded_file.status = "Failed"
    uploaded_file.save()
    AuditLog.objects.create(
        user=request.user,
        action_type="Upload Error",
        details=f"Failed to process {file.name}: {str(e)}",
    )
    messages.error(request, f"{file.name} failed to process.")

```

```
request.session["parsed_files"] = parsed_files

if success_count > 0:
    messages.success(request, "File(s) processed successfully.")

    return redirect("upload_file")

return render(
    request,
    "upload.html",
    {
        "latest_uploads": latest_uploads,
        "domains": domains,
        "parsed_files": request.session.get("parsed_files"),
    },
)
```

6.4.2 Excel Data Validation Logic

```
def validate_excel_data(uploaded_file_obj):
    import pandas as pd
    from .models import ValidationRule, FormulaRule, ValidationResult

    file_path = uploaded_file_obj.file.path
    df = (
        pd.read_csv(file_path)
        if file_path.endswith(".csv")
        else pd.read_excel(file_path)
    )
    df = df.loc[:, ~df.columns.astype(str).str.contains("^Unnamed")]

    template = uploaded_file_obj.template
    rules = ValidationRule.objects.filter(template=template)
    formula_rules = FormulaRule.objects.filter(template=template)
    ValidationResult.objects.filter(file=uploaded_file_obj).delete()

    for index, row in df.iterrows():
        failed_columns_for_row = set()

        row_context = {}
        saved_mappings = getattr(uploaded_file_obj, "column_mappings", {})

        for key, val_str in row.items():
            if pd.isna(key) or str(key).strip() == "":
                continue

            if pd.isna(val_str) or val_str == "":
                parsed_val = 0
            else:
                raw_str = str(val_str).strip()
                try:
                    parsed_val = float(raw_str)
                except:
                    parsed_val = raw_str

            clean_key = str(key).replace(" ", "")
            row_context[clean_key] = parsed_val
            row_context[clean_key.title()] = parsed_val

            if key in saved_mappings:
                db_name = saved_mappings[key]
```

```

        row_context[db_name] = parsed_val

    for rule in rules:
        actual_col = find_best_column(rule.column_name, df.columns)
        if actual_col:
            raw_val = row[actual_col]

            if pd.isna(raw_val) or raw_val == "":
                val = 0 if ("<" in rule.condition_expression or ">" in
rule.condition_expression) else ""
            else:
                try:
                    if "date" in str(actual_col).lower():
                        val = pd.to_datetime(raw_val, errors="coerce")
                    else:
                        val = float(raw_val)
                except:
                    val = str(raw_val).strip()

            try:
                allowed_locals = {"x": val, "index": index}
                allowed_locals.update(row_context)

                allowed_globals = {
                    "__builtins__": None,
                    "str": str, "int": int, "float": float,
                    "len": len, "abs": abs, "round": round,
                }

                if not eval(rule.condition_expression, allowed_globals,
allowed_locals):
                    ValidationResult.objects.create(
                        file=uploaded_file_obj,
                        row_index=index + 1,
                        column_name=actual_col,
                        error_details=rule.error_message,
                        is_valid=False,
                    )
                    failed_columns_for_row.add(actual_col)
            except Exception as e:
                print(f"Logic Error: {e}")
                continue

    for f_rule in formula_rules:
        actual_target_col = find_best_column(f_rule.target_column,

```

```

df.columns)

    if actual_target_col and actual_target_col not in
failed_columns_for_row:
    try:
        excel_val = row[actual_target_col]

        safe_formula_context = {
            k: (0 if v == "" else v)
            for k, v in row_context.items()
        }

        expected_val = eval(
            f_rule.condition_expression,
            {"__builtins__": None},
            safe_formula_context
        )

        display_excel_val = "Empty" if pd.isna(excel_val) or
str(excel_val).strip() == "" else str(excel_val).strip()
        display_expected_val = str(expected_val).strip()

        if display_excel_val != display_expected_val:
            ValidationResult.objects.create(
                file=uploaded_file_obj,
                row_index=index + 1,
                column_name=actual_target_col,
                error_details=f"Math Error: Expected
{display_expected_val}, found {display_excel_val}.",
                is_valid=False,
            )
    except Exception as e:
        print(f"Formula Error: {e}")
        continue

```

6.4.3 AI Formula Explanation

```
def generate_ai_explanation(
    formula_name, target_column, condition_expression, context_data=None
):
    ai_api_key = os.environ.get("AI_API_KEY")
    ai_api_url = os.environ.get(
        "AI_API_URL", "https://api.openai.com/v1/chat/completions"
    )
    ai_model = os.environ.get("AI_MODEL", "gpt-3.5-turbo")

    if ai_api_key:
        try:
            prompt = (
                f"You are a data analyst explaining spreadsheet formulas in "
                f"simple, clear terms. "
                f"Be concise and use bullet points for different aspects. "
                f"Now, explain this validation error in simple terms:\n"
                f"Rule: {formula_name}\n"
                f"Column: {target_column}\n"
                f"Condition: {condition_expression}"
            )
            if context_data:
                prompt += f"\nContext: {context_data}"

            response = requests.post(
                ai_api_url,
                headers={
                    "Authorization": f"Bearer {ai_api_key}",
                    "Content-Type": "application/json",
                },
                json={
                    "model": ai_model,
                    "messages": [{"role": "user", "content": prompt}],
                    "max_tokens": 200,
                },
                timeout=10,
            )
            response.raise_for_status()
            data = response.json()
            return data["choices"][0]["message"]["content"].strip(), True
        except Exception as e:
            print(f"AI API call failed: {e}")
```

Fallback: rule-based explanation

```

    if "Math Error" in condition_expression:
        explanation = (
            f"***Logic Mismatch Detected:** The value in `{target_column}`
doesn't align with the "
            f"calculated results for `{formula_name}`. The system found a
manual discrepancy "
            "against the standard formula."
        )
    elif any(
        x in condition_expression.lower()
        for x in ["exceed", "between", "limit", "<=", ">="]
    ):
        numbers = re.findall(r"\d+", condition_expression)
        if numbers:
            max_val = max(map(int, numbers))
            min_val = min(map(int, numbers)) if len(numbers) > 1 else 0
            explanation = (
                f"***Boundary Violation:** The entry for `{target_column}` is
invalid. "
                f"The value must be between **{min_val} and {max_val}**."
            )
        else:
            explanation = f"***Boundary Violation:** The value in
`${target_column}` exceeds the limit."
    else:
        explanation = (
            f"***System Validation:** The `${target_column}` field failed the
`${formula_name}` "
            f"verification. Logic checked: *{condition_expression}*."
        )

    if context_data:
        explanation += f"\n\n***Context Found:** " + ", ".join(
            f"`{k}={v}`" for k, v in context_data.items()
        )

    return explanation.strip(), False

```

6.4.4 PDF/TXT Text Extraction

```
def extract_text_from_pdf(pdf_file):
    """Extract text from uploaded PDF file."""
    reader = PyPDF2.PdfReader(pdf_file)
    text = ""
    for page in reader.pages:
        page_text = page.extract_text()
        if page_text:
            text += page_text + "\n"
    return text

def call_ai_to_csv(raw_text):
    """Send raw text to AI and ask for CSV output."""
    if not AI_API_KEY:
        raise Exception("AI_API_KEY not set in environment")

    prompt = f"""You are a data extraction assistant. Convert the following
text into a structured CSV table.
- Detect column headers automatically.
- Ensure each row has the same number of columns.
- Output ONLY valid CSV data, no extra text, no markdown.
- Use double quotes for fields that contain commas or newlines.

Text:
{raw_text[:3000]}
"""

    headers = {
        "Authorization": f"Bearer {AI_API_KEY}",
        "Content-Type": "application/json",
    }
    payload = {
        "model": AI_MODEL,
        "messages": [{"role": "user", "content": prompt}],
        "temperature": 0.2,
    }

    response = requests.post(AI_API_URL, headers=headers, json=payload,
timeout=30)
    response.raise_for_status()
    data = response.json()
    csv_content = data["choices"][0]["message"]["content"].strip()

    if csv_content.startswith("`"):

```

```

    lines = csv_content.splitlines()
    if lines[0].startswith("`"):
        lines = lines[1:]
    if lines and lines[-1].startswith("`"):
        lines = lines[:-1]
    csv_content = "\n".join(lines)

    return csv_content

def fallback_parse_text_to_rows(raw_text):
    """Rule-based fallback when AI fails."""
    rows = []
    for line in raw_text.strip().splitlines():
        if line.strip():
            cols = line.split()
            if cols:
                rows.append(cols)

    if len(rows) == 1:
        rows = [[cell] for cell in rows[0]]

    return rows if rows else None

```

6.4.5 Column Type Detection

```
def detect_column_types(df):
    detected_types = {}
    for column in df.columns:
        sample = df[column].dropna()
        if sample.empty:
            detected_types[column] = "Empty"
            continue

        total = len(sample)

        # Numeric check
        numeric_converted = pd.to_numeric(sample, errors="coerce")
        numeric_ratio = numeric_converted.notna().sum() / total
        if numeric_ratio > 0.8:
            detected_types[column] = "Numeric"
            continue

        # Date pattern check
        sample_str = sample.astype(str)
        has_date_pattern = sample_str.str.contains(
            r'[/\-\_]|jan|feb|mar|apr|may|jun|jul|aug|sep|oct|nov|dec',
            case=False, na=False
        ).any()

        date_ratio = 0
        if has_date_pattern:
            try:
                date_converted = pd.to_datetime(sample, errors='coerce',
                    dayfirst=True)
                date_ratio = date_converted.notna().sum() / total
            except:
                pass

        if date_ratio < 0.4:
            for fmt in ["%Y-%m-%d", "%d/%m/%Y", "%Y/%m/%d"]:
                try:
                    date_converted = pd.to_datetime(sample, format=fmt,
                        errors='coerce')
                    ratio = date_converted.notna().sum() / total
                    if ratio > date_ratio:
                        date_ratio = ratio
                    if date_ratio >= 0.4:
                        break
```

```
        except:
            continue

    # Final classification
    if date_ratio >= 0.4:
        detected_types[column] = "Date"
    elif numeric_ratio > 0.2:
        detected_types[column] = "Mixed"
    else:
        detected_types[column] = "Text"

return detected_types
```

6.4.6 Quality Score Calculation

```
def calculate_quality_score(uploaded_file_obj, total_rows):  
    from .models import ValidationResult  
  
    error_rows_count = (  
        ValidationResult.objects.filter(file=uploaded_file_obj, is_valid=False)  
        .values("row_index")  
        .distinct()  
        .count()  
    )  
  
    if total_rows == 0:  
        return 0  
  
    success_rows = total_rows - error_rows_count  
    score = (success_rows / total_rows) * 100  
    return round(score, 2)
```

6.4.7 Excel Preview with Styling

```
@login_required
def preview_excel_view(request, file_id):
    uploaded_file = get_object_or_404(UploadedFile, id=file_id,
user=request.user)
    file_path = uploaded_file.file.path

    if file_path.endswith(".csv"):
        df = pd.read_csv(file_path)
    else:
        df = pd.read_excel(file_path)

    df = preprocess_dataframe(df)
    df = remove_empty_unnamed_columns(df)
    df.columns = [str(col).replace("_", " ").title() for col in df.columns]
    df = df.fillna("")

    style_data = getattr(uploaded_file, "style_data", None)

    if isinstance(style_data, str) and style_data.strip():
        try:
            style_data = json.loads(style_data)
        except Exception:
            try:
                style_data = ast.literal_eval(style_data)
            except Exception:
                style_data = None

    html = '<table class="excel-table">'
    html += '<thead><tr>'
    for col in df.columns:
        html += f'<th>{col}</th>'
    html += '</tr></thead><tbody>'

    for i, row in enumerate(df.values):
        html += '<tr>'
        for j, val in enumerate(row):
            style_attr = ""
            if style_data and i < len(style_data) and j < len(style_data[i]):
                cell = style_data[i][j]
                if isinstance(cell, dict):
                    styles = []
                    if cell.get("bold"):
                        styles.append("font-weight:bold")
                    if cell.get("italic"):
```

```

        styles.append("font-style:italic")
    if cell.get("color"):
        styles.append(f"color:{cell.get('color')}")
    if cell.get("bg"):
        styles.append(f"background:{cell.get('bg')}")
    if cell.get("fontFamily"):
        styles.append(f"font-family:{cell.get('fontFamily')}")
    if cell.get("fontSize"):
        styles.append(f"font-size:{cell.get('fontSize')}")
    style_attr = f' style="{"".join(styles)}"'
    html += f'<td{style_attr}>{val}</td>'
    html += '</tr>'

html += '</tbody></table>'

return JsonResponse({"table": html})

```

6.4.8 Workbook Editor – Load File with Formulas, Styles, and Mappings

```
@login_required
def workbook_editor_view(request, file_id):
    uploaded_file = get_object_or_404(UploadedFile, id=file_id,
user=request.user)
    template = uploaded_file.template
    file_path = uploaded_file.file.path

    val_cols =
list(ValidationRule.objects.filter(template=template).values_list("column_name",
flat=True))
    form_cols =
list(FormulaRule.objects.filter(template=template).values_list("target_column",
flat=True))
    db_columns = list(set(val_cols + form_cols))

    def normalize(col):
        col = str(col).lower()
        col = re.sub(r"[ _()%.\\-\\/]", "", col)
        return col

    def find_matching_db_column(file_col, db_columns):
        file_norm = normalize(file_col)
        for db_col in db_columns:
            db_norm = normalize(db_col)
            if db_norm == file_norm:
                return db_col
            if db_norm in file_norm or file_norm in db_norm:
                return db_col
            similarity = difflib.SequenceMatcher(None, db_norm,
file_norm).ratio()
            if similarity >= 0.65:
                return db_col
        return None

    saved_mappings = getattr(uploaded_file, "column_mappings", {})

    try:
        file_ext = os.path.splitext(file_path)[1].lower()
        is_csv = file_ext == ".csv"

        if is_csv:
            df = pd.read_csv(file_path, dtype=str)
            df = df.fillna("")
```

```

        headers = list(df.columns)
        raw_values = df.values.tolist()
        raw_formulas = [[None for _ in headers] for _ in raw_values]
    else:
        from openpyxl import load_workbook
        wb_formula = load_workbook(file_path, data_only=False)
        ws_formula = wb_formula.active
        wb_value = load_workbook(file_path, data_only=True)
        ws_value = wb_value.active

        headers = []
        for cell_f, cell_v in zip(next(ws_formula.iter_rows(min_row=1,
max_row=1)),
                                next(ws_value.iter_rows(min_row=1,
max_row=1))):
            headers.append(cell_v.value if cell_v.value is not None else "")

        raw_values = []
        raw_formulas = []
        for row_f, row_v in zip(ws_formula.iter_rows(min_row=2),
ws_value.iter_rows(min_row=2)):
            val_row = []
            formula_row = []
            for cell_f, cell_v in zip(row_f, row_v):
                val_row.append(cell_v.value if cell_v.value is not None else
"")

                if isinstance(cell_f.value, str) and
cell_f.value.startswith('='):
                    formula_row.append(cell_f.value)
                else:
                    formula_row.append(None)
            raw_values.append(val_row)
            raw_formulas.append(formula_row)

    df = pd.DataFrame(raw_values, columns=headers)
    formulas_df = pd.DataFrame(raw_formulas, columns=headers)

    # 1. Remove unnamed and fully empty columns
    unnamed_mask = df.columns.astype(str).str.lower().str.contains("unnamed")
    empty_cols = (df.isna().sum() == len(df))
    cols_to_drop = unnamed_mask & empty_cols
    df = df.loc[:, ~cols_to_drop]
    formulas_df = formulas_df.loc[:, ~cols_to_drop]

```

```

new_cols = [str(c).strip().lower().replace(" ", "_") for c in df.columns]
df.columns = new_cols
formulas_df.columns = new_cols

# 3. Trim string values in df
for col in df.columns:
    if pd.api.types.is_object_dtype(df[col]):
        df[col] = df[col].astype(str).str.strip()
        df[col] = df[col].replace("nan", pd.NA)

# 4. Remove rows that are completely empty (all NaN)
empty_rows = df.isna().all(axis=1)
empty_rows_arr = empty_rows.values
df = df[~empty_rows_arr]
formulas_df = formulas_df[~empty_rows_arr]

# 5. Remove rows with only one non-empty value
row_non_na = df.count(axis=1).values # numpy array
rows_to_keep = row_non_na > 1
df = df[rows_to_keep]
formulas_df = formulas_df[rows_to_keep]

df = df.reset_index(drop=True)
formulas_df = formulas_df.reset_index(drop=True)

user_columns_original = list(df.columns)

columns_with_classes = []
ai_matches = {}
is_first_load = not bool(saved_mappings) # Keep track if this is the
first load
new_initial_mapping = {} # Store AI mapping to save to DB

if is_first_load:
    ai_matches = ai_match_columns(user_columns_original, db_columns)

for col in user_columns_original:
    matched_db = None
    if saved_mappings and col in saved_mappings:
        matched_db = saved_mappings[col]
    elif not saved_mappings and ai_matches.get(col):
        matched_db = ai_matches[col]
    else:
        matched_db = find_matching_db_column(col, db_columns)

```

```

        if matched_db:
            columns_with_classes.append({"name": col, "class": "header-
matched", "matched_db": matched_db})
            if is_first_load:
                new_initial_mapping[col] = matched_db # Capture the new
mapping
            else:
                columns_with_classes.append({"name": col, "class": "header-
custom", "matched_db": None})

        matched_cols = [c["matched_db"] for c in columns_with_classes if
c["matched_db"]]

        is_edited_file = "edited" in uploaded_file.file.name.lower()

        if not is_edited_file:
            for db_col in db_columns:
                # If a required template column is missing from the user's file
                if db_col not in matched_cols and db_col not in df.columns:
                    df[db_col] = ""
                    formulas_df[db_col] = None
                    columns_with_classes.append({
                        "name": db_col,
                        "class": "header-template-only",
                        "matched_db": db_col
                    })

        if is_first_load and new_initial_mapping:
            uploaded_file.column_mappings = new_initial_mapping
            uploaded_file.save()
            saved_mappings = new_initial_mapping # Update variable so JS gets the
data

        current_columns = [c["name"] for c in columns_with_classes]
        preview_data = df.fillna("").values.tolist()
        formulas_data = formulas_df.fillna("").values.tolist()

        current_columns = [c["name"] for c in columns_with_classes]
        preview_data = df.fillna("").values.tolist()
        formulas_data = formulas_df.fillna("").values.tolist()

    except Exception as e:
        import traceback
        traceback.print_exc()
        return HttpResponse(f"Error loading file: {e}")

```

```

saved_mappings_json = json.dumps(saved_mappings)
style_data_raw = getattr(uploaded_file, "style_data", [])
if isinstance(style_data_raw, str) and style_data_raw:
    try:
        style_data = json.loads(style_data_raw)
    except:
        style_data = []
else:
    style_data = style_data_raw if style_data_raw else []
style_data_json = json.dumps(style_data)
formulas_data_json = json.dumps(formulas_data)

validate_excel_data(uploaded_file)
errors = ValidationResult.objects.filter(file=uploaded_file, is_valid=False)
errors_data = [
    {
        "row": err.row_index,
        "column": err.column_name,
        "message": err.error_details,
        "id": err.id,
    }
    for err in errors
]

recommendations = get_formula_recommendations(uploaded_file, df.columns)
print("RECOMMENDATIONS FROM BACKEND:", recommendations)

return render(request, "workbook_editor.html", {
    "file": uploaded_file,
    "current_columns": current_columns,
    "columns_with_classes": columns_with_classes,
    "db_columns": db_columns,
    "user_columns": user_columns_original,
    "preview_data": preview_data,
    "saved_mappings_json": saved_mappings_json,
    "style_data_json": style_data_json,
    "formulas_data_json": formulas_data_json,
    "errors_data_json": json.dumps(errors_data),
    "recommendations_json": json.dumps(recommendations),
    "is_draft": uploaded_file.status != "Completed"
})

```

6.4.9 Real-time Cell Validation (Live Error Highlight)

```
@csrf_exempt
@login_required
def validate_single_cell(request):
    if request.method != 'POST':
        return JsonResponse({'error': 'POST required'}, status=400)

    data = json.loads(request.body)
    file_id = data.get('file_id')
    column_name = data.get('column')
    value = data.get('value')
    row_index = (data.get('row') or 1) - 1
    row_data = data.get('rowData', {})

    uploaded_file = get_object_or_404(UploadedFile, id=file_id,
user=request.user)
    template = uploaded_file.template

    saved_mappings = getattr(uploaded_file, "column_mappings", {})

    from .models import ValidationRule
    rules = ValidationRule.objects.filter(template=template)

    def normalize(s):
        return re.sub(r'[_()%\-/]', '', str(s)).lower()

    target_norm = normalize(column_name)
    matched_rule = None

    # 1. Try direct rule match by column name
    for rule in rules:
        if normalize(rule.column_name) == target_norm:
            matched_rule = rule
            break

    # 2. If not found, check if this file column is mapped to a DB column that
has a rule
    if not matched_rule:
        for file_col, db_col in saved_mappings.items():
            if normalize(file_col) == target_norm:
                for rule in rules:
                    if normalize(rule.column_name) == normalize(db_col):
                        matched_rule = rule
                        break
        if matched_rule:
```

```

        break

    if not matched_rule:
        return JsonResponse({'is_valid': True})

    try:
        if value and str(value).replace('.', '', 1).replace('-', '',
1).isdigit():
            val = float(value)
        else:
            val = str(value).strip() if value is not None else ""

    allowed_locals = {'x': val, 'index': row_index}

    for key, val_str in row_data.items():
        if not key:
            continue
        clean_key = str(key).replace(" ", "")

        # Parse numeric values
        if val_str is not None and str(val_str).replace('.', '',
1).replace('-', '', 1).isdigit():
            parsed_val = float(val_str)
        else:
            parsed_val = str(val_str).strip() if val_str else ""
            if parsed_val == "" and ("<" in matched_rule.condition_expression
or ">" in matched_rule.condition_expression):
                parsed_val = 0

        allowed_locals[clean_key] = parsed_val
        allowed_locals[clean_key.lower()] = parsed_val
        allowed_locals[clean_key.upper()] = parsed_val
        allowed_locals[clean_key.title()] = parsed_val
        allowed_locals[clean_key.capitalize()] = parsed_val

    for excel_col, db_col in saved_mappings.items():
        if normalize(excel_col) == normalize(key):
            allowed_locals[db_col] = parsed_val
            allowed_locals[str(db_col).replace(" ", "")] = parsed_val

    allowed_globals = {
        '__builtins__': None,
        'str': str, 'int': int, 'float': float,
        'len': len, 'abs': abs, 'round': round,
        'isinstance': isinstance, 'type': type

```

```

    }

    condition = matched_rule.condition_expression.strip()
    is_valid = bool(eval(condition, allowed_globals, allowed_locals))
    error_msg = matched_rule.error_message if not is_valid else ''

    except Exception as e:
        is_valid = False
        error_msg = f'Formula Error: {str(e)}'

    return JsonResponse({
        'is_valid': is_valid,
        'error_message': error_msg,
    })

```

6.4.10 Formula Evaluation Engine (JavaScript – COUNTIF with Quote Stripping)

```

if (func === 'COUNTIF') {
    if (args.length < 2) return '#ERROR';
    let range = args[0];
    let criteria = args[1];
    let cells = getCellsInRange(range, getCellValue);
    let count = 0;

    let cleanCriteria = criteria;
    if (typeof cleanCriteria === 'string') {
        cleanCriteria = cleanCriteria.trim();
        if ((cleanCriteria.startsWith('"') && cleanCriteria.endsWith('"')) ||
            (cleanCriteria.startsWith("'") && cleanCriteria.endsWith("'"))) {
            cleanCriteria = cleanCriteria.slice(1, -1);
        }
    }
    console.log("COUNTIF criteria (cleaned):", cleanCriteria);

    for (let cell of cells) {
        let val = getCellValue(cell);
        let match = false;

        if (cleanCriteria.startsWith('>') || cleanCriteria.startsWith('<') ||
            cleanCriteria.startsWith('=')) {
            let op = cleanCriteria.match(/[<>]=?/)[0];
            let target = parseFloat(cleanCriteria.slice(op.length));
            let num = parseFloat(val);
            if (isNaN(num)) continue;
            if (op === '>' && num > target) match = true;

```

```

        else if (op === '<' && num < target) match = true;
        else if (op === '>=' && num >= target) match = true;
        else if (op === '<=' && num <= target) match = true;
        else if (op === '=' && num == target) match = true;
    } else {

        let cleanVal = (typeof val === 'string') ? val.trim() : val;
        let cleanCrit = (typeof cleanCriteria === 'string') ?
cleanCriteria.trim() : cleanCriteria;
        match = (cleanVal == cleanCrit);
    }
    if (match) count++;
}
return count;
}

console.log("COUNTIF criteria (cleaned):", cleanCriteria);

for (let cell of cells) {
    let val = getCellValue(cell);
    let match = false;

    if (cleanCriteria.startsWith('>') || cleanCriteria.startsWith('<') ||
cleanCriteria.startsWith('=')) {
        let op = cleanCriteria.match(/[<>]=?/)[0];
        let target = parseFloat(cleanCriteria.slice(op.length));
        let num = parseFloat(val);
        if (isNaN(num)) continue;
        if (op === '>' && num > target) match = true;
        else if (op === '<' && num < target) match = true;
        else if (op === '>=' && num >= target) match = true;
        else if (op === '<=' && num <= target) match = true;
        else if (op === '=' && num == target) match = true;
    } else {

        let cleanVal = (typeof val === 'string') ? val.trim() : val;
        let cleanCrit = (typeof cleanCriteria === 'string') ?
cleanCriteria.trim() : cleanCriteria;
        match = (cleanVal == cleanCrit);
    }
    if (match) count++;
}
return count;
}

```

6.4.11 Smart Range Detection (JavaScript)

```
function getSmartRange(cell) {
  const table = document.getElementById('tableBody');
  const rows = Array.from(table.rows);
  const cellIndex = cell.cellIndex;
  const rowIndex = cell.parentElement.rowIndex - 1;

  const hasValue = (r, c) => {
    if (!rows[r] || !rows[r].cells[c]) return false;
    let val = rows[r].cells[c].innerText.trim();
    return val !== "";
  };

  // Vertical: cells above and below
  let topRow = rowIndex - 1;
  let hasAbove = false;
  while (topRow >= 0 && hasValue(topRow, cellIndex)) {
    hasAbove = true;
    topRow--;
  }
  let bottomRow = rowIndex + 1;
  let hasBelow = false;
  while (bottomRow < rows.length && hasValue(bottomRow, cellIndex)) {
    hasBelow = true;
    bottomRow++;
  }
  if (hasAbove || hasBelow) {
    let startRow = hasAbove ? topRow + 1 : rowIndex + 1;
    let endRow = hasBelow ? bottomRow - 1 : rowIndex - 1;
    if (startRow <= endRow) {
      return `${numToCol(cellIndex - 1)}${startRow +
1}:${numToCol(cellIndex - 1)}${endRow + 1}`;
    }
  }

  // Horizontal: cells left and right
  let leftCol = cellIndex - 1;
  let hasLeft = false;
  while (leftCol >= 1 && hasValue(rowIndex, leftCol)) {
    hasLeft = true;
    leftCol--;
  }
  let rightCol = cellIndex + 1;
  let hasRight = false;
```

```

const maxCol = rows[0] ? rows[0].cells.length - 1 : cellIndex;
while (rightCol <= maxCol && hasValue(rowIndex, rightCol)) {
  hasRight = true;
  rightCol++;
}
if (hasLeft || hasRight) {
  let startCol = hasLeft ? leftCol + 1 : cellIndex + 1;
  let endCol = hasRight ? rightCol - 1 : cellIndex - 1;
  if (startCol <= endCol) {
    return `${numToCol(startCol - 1)}${rowIndex + 1}:${numToCol(endCol - 1)}${rowIndex + 1}`;
  }
}

return "";
}

```

6.4.12 Undo/Redo State Capture (JavaScript)

```

function captureState() {
  if (isRestoring) return;
  try {
    const headers = Array.from(document.querySelectorAll('#headerRow th:not(.row-header-cell)')).map(th => {
      const select = th.querySelector('.mapping-select');
      const label = th.querySelector('.header-label');
      return {
        outerHTML: th.outerHTML,
        text: label ? label.innerText.trim() : '',
        selectedValue: select ? select.value : ''
      };
    });

    const rows = [];
    document.querySelectorAll('#tableBody tr').forEach(tr => {
      const rowData = [];
      tr.querySelectorAll('td.editable-cell').forEach(td => {

        let rawText = '';
        if (td) {
          rawText = Array.from(td.childNodes)
            .filter(node => node.nodeType === Node.TEXT_NODE)
            .map(node => node.textContent)
            .join('');

```

```

        .trim();
    }

    rowData.push({
        text: rawText,
        style: td ? td.style.cssText : '',
        formula: (td && cellFormulas.has(td)) ? cellFormulas.get(td)
: '',
        hasError: td ? td.classList.contains('validation-error') :
false,
        errorMsg: td ? td.getAttribute('data-error') : '',
        errorId: td ? td.getAttribute('data-error-id') : ''
    });
});
rows.push(rowData);
});

if (headers.length === 0 || rows.length === 0) return;

const currentState = { headers: headers, rows: rows };
const currentStateJson = JSON.stringify(currentState);

if (currentStateJson !== lastStateJson) {
    historyStack.push(currentState);
    lastStateJson = currentStateJson;
    redoStack = [];
    if (historyStack.length > 50) historyStack.shift();
}
} catch (e) {
    console.warn("Capture state error:", e);
}
}

```

6.4.13 Editor Initialization (Restore Errors & Recommendations)

```

function init() {
    attachCellEvents();
    updateStatusBar();
    renumberRows();
    attachKeyboardNavigation();
    setupFormattingToolbar();
    const uiBtns = { 'addRowBtn': addRow, 'addColBtn': addColumn, 'undoBtn':
undo, 'redoBtn': redo, 'saveProcessBtn': saveData };
    Object.entries(uiBtns).forEach(([id, fn]) => { const el =
document.getElementById(id); if (el) el.onclick = fn; });
}

```

```

let saved = window.DJANGO_VARS.savedMappings;
while (typeof saved === 'string') {
  try { saved = JSON.parse(saved); } catch(e) { break; }
}
if (saved && typeof saved === 'object') {
  document.querySelectorAll('#headerRow th:not(.row-header-cell)').forEach(th => {
    const label = th.querySelector('.header-label')?.innerText.trim();
    if (saved[label]) {
      const s = th.querySelector('.mapping-select');
      if (s) { s.value = saved[label]; handleMappingChange(s); }
    }
  });
}
let styles = window.DJANGO_VARS.styleData;
while (typeof styles === 'string') {
  try { styles = JSON.parse(styles); } catch(e) { break; }
}
if (styles && Array.isArray(styles)) {
  const trs = document.querySelectorAll('#tableBody tr');
  styles.forEach((row, ri) => {
    if (trs[ri]) {
      const tds = trs[ri].querySelectorAll('td.editable-cell');
      row.forEach((c, ci) => {
        if (tds[ci]) {
          if (c.bold) tds[ci].style.fontWeight = 'bold';
          if (c.italic) tds[ci].style.fontStyle = 'italic';
          if (c.underline === 'double')
tds[ci].style.textDecoration = 'underline double';
          else if (c.underline === 'single' || c.underline ===
true) tds[ci].style.textDecoration = 'underline';
          if (c.color) tds[ci].style.color = c.color;
          if (c.bg) tds[ci].style.backgroundColor = c.bg;
          if (c.fontFamily) tds[ci].style.fontFamily =
c.fontFamily;
          if (c.fontSize) tds[ci].style.fontSize = c.fontSize;
        }
      });
    }
  });
}
let formulas = window.DJANGO_VARS.formulasData;
while (typeof formulas === 'string') {
  try { formulas = JSON.parse(formulas); } catch(e) { break; }
}

```

```

if (formulas && Array.isArray(formulas)) {
  const trs = document.querySelectorAll('#tableBody tr');
  formulas.forEach((row, ri) => {
    if (trs[ri]) {
      const tds = trs[ri].querySelectorAll('td.editable-cell');
      row.forEach((formula, ci) => {
        if (formula && tds[ci] && typeof formula === 'string' &&
formula.startsWith('=')) {
          cellFormulas.set(tds[ci], formula);
        }
      });
    }
  });
}

if (activeCell) updateValueDisplayForCell(activeCell);
const formulaInput = document.getElementById('formulaInput');
if (formulaInput) {
  formulaInput.addEventListener('input', (e) =>
window.showFormulaHint(e.target.value));
}

const formulaDropdown = document.getElementById('formulaDropdown');
if (formulaDropdown) {
  formulaDropdown.addEventListener('change', function() {
    const selected = this.value;
    if (selected && activeCell) {
      const smartRange = getSmartRange(activeCell);
      const fInput = document.getElementById('formulaInput');
      if (smartRange) {
        fInput.value = `=${selected}(${smartRange})`;
      } else {
        fInput.value = `=${selected}`;
      }
      showFormulaHint(fInput.value);
      fInput.focus();
      updateCellFromFormulaBar();
      this.selectedIndex = 0;
    }
  });
}

// Floating formula panel
const formulaBtn = document.getElementById('formulaPopupBtn');
const formulaPanel = document.getElementById('formulaPanel');

if (formulaBtn && formulaPanel) {

```

```

formulaBtn.addEventListener('click', (e) => {
    e.stopPropagation();
    const rect = formulaBtn.getBoundingClientRect();
    let top = rect.bottom + window.scrollY + 5;
    let left = rect.left + window.scrollX;
    const panelWidth = formulaPanel.offsetWidth;
    const panelHeight = formulaPanel.offsetHeight;
    const viewportWidth = window.innerWidth;
    const viewportHeight = window.innerHeight;

    if (left + panelWidth > viewportWidth + window.scrollX) {
        left = viewportWidth + window.scrollX - panelWidth - 10;
    }
    if (left < window.scrollX) {
        left = window.scrollX + 10;
    }
    if (top + panelHeight > window.scrollY + viewportHeight) {
        top = rect.top + window.scrollY - panelHeight - 5;
    }
    formulaPanel.style.top = top + 'px';
    formulaPanel.style.left = left + 'px';
    formulaPanel.style.display = 'block';
});

document.addEventListener('click', (e) => {
    if (!formulaPanel.contains(e.target) && e.target !== formulaBtn) {
        formulaPanel.style.display = 'none';
    }
});

document.querySelectorAll('.panel-formula-btn').forEach(btn => {
    btn.addEventListener('click', (e) => {
        e.preventDefault();
        e.stopPropagation();
        if (!activeCell) return;
        const func = btn.getAttribute('data-func');
        const smartRange = getSmartRange(activeCell);
        const fInput = document.getElementById('formulaInput');
        if (fInput) {
            if (smartRange) {
                fInput.value = `=${func.toUpperCase()}(${smartRange})`;
            } else {
                fInput.value = `=${func.toUpperCase()}(`;
            }
            if (typeof window.showFormulaHint === 'function') {

```

```

        window.showFormulaHint(fInput.value);
    }
    updateCellFromFormulaBar();
    activeCell.focus();
}
formulaPanel.style.display = 'none';
});
});
}

// --- Validation error highlighting ---
if (window.DJANGO_VARS.errorsData && window.DJANGO_VARS.errorsData.length) {
    // Use a Map to store only one button per cell
    const cellToError = new Map();
    window.DJANGO_VARS.errorsData.forEach(err => {
        const row = err.row - 1;
        const errCol = err.column.trim().toLowerCase();
        const headers = Array.from(document.querySelectorAll('#headerRow
th:not(.row-header-cell)'));
        const normalize = s => s.replace(/\s_()%\-/g, '').toLowerCase();
        const colIndex = headers.findIndex(th => {
            const label = th.querySelector('.header-label');
            if (!label) return false;
            const headerText = label.innerText.trim().toLowerCase();
            return normalize(headerText) === normalize(errCol);
        });
        if (colIndex >= 0) {
            const cell = document.querySelector(`#tableBody tr:nth-
child(${row+1}) td.editable-cell:nth-child(${colIndex+2})`);
            if (cell && !cellToError.has(cell)) {
                cellToError.set(cell, err);
                cell.classList.add('validation-error');
                cell.setAttribute('data-error', err.message);
                cell.setAttribute('data-error-id', err.id);
                const explainBtn = document.createElement('button');
                explainBtn.textContent = 'AI Explain';
                explainBtn.className = 'ai-explain-btn';
                explainBtn.setAttribute('data-result-id', err.id);
                explainBtn.setAttribute('data-row', err.row);
                explainBtn.setAttribute('data-column', err.column);
                explainBtn.setAttribute('title', 'Get AI explanation');
                explainBtn.contentEditable = 'false';
                explainBtn.style.display = 'none';
                explainBtn.onclick = (e) => {
                    e.stopPropagation();

```

```

    fetch('/ai-explain/', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json', 'X-CSRFToken':
window.DJANGO_VARS.csrfToken },
      body: JSON.stringify({ result_id: err.id })
    })
    .then(res => res.json())
    .then(data => {
      const panel = document.createElement('div');
      panel.className = 'floating-explanation-panel';
      panel.innerHTML = `
        <div class="ai-explanation-box">
          <div class="ai-insight-header">💡 Intelligent Auditor
Analysis</div>
          <div class="ai-content">${markdownToHtml(data.explanation)}</div>
        </div>
        <button class="close-explanation">✕</button>
      `;
      document.body.appendChild(panel);
      panel.querySelector('.close-explanation').onclick = () => panel.remove();
      setTimeout(() => panel.remove(), 8000);
    })
    .catch(err => console.error(err));
  });

  cell.appendChild(explainBtn);
  cell.addEventListener('mouseenter', () =>
explainBtn.style.display = 'inline-block');
  cell.addEventListener('mouseleave', () =>
explainBtn.style.display = 'none');
}
}
});
}

console.log("DEBUG: recommendations data:", window.DJANGO_VARS.recommendations);
console.log("DEBUG: recommendations length:", window.DJANGO_VARS.recommendations
? window.DJANGO_VARS.recommendations.length : 'undefined');

// --- Recommendations panel (always visible, placed after status bar) ---
const targetContainer = document.getElementById('recommendationsTarget');
if (targetContainer) {
  const panel = document.createElement('div');
  panel.id = 'recommendationsPanel';
  panel.className = 'recommendations-wrapper';

```

```

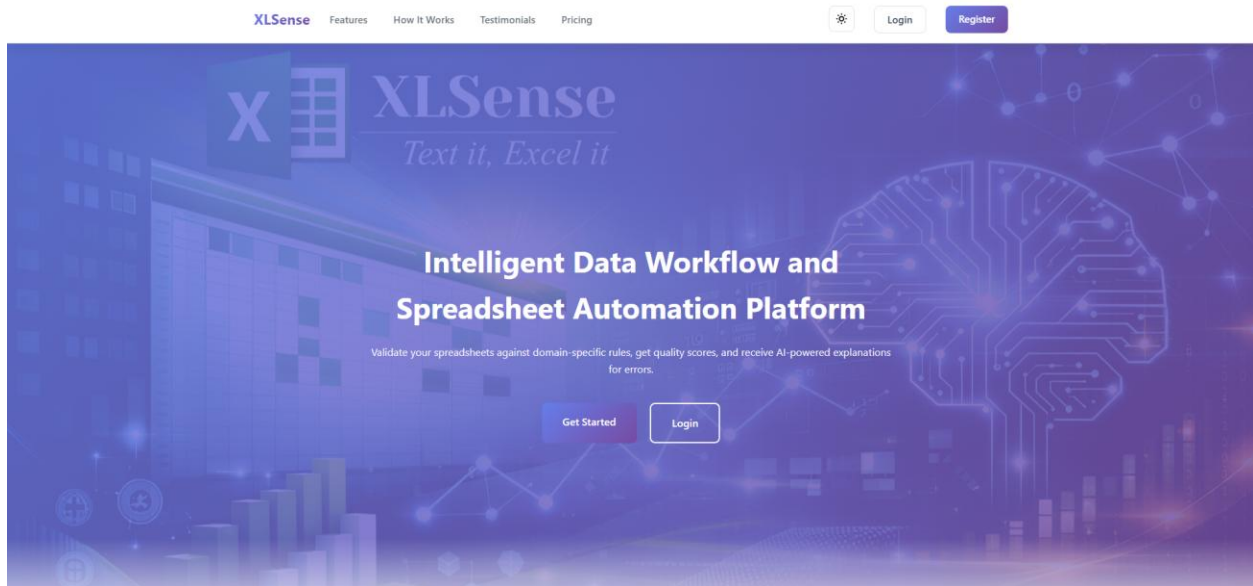
    panel.innerHTML = `
        <h3 class="section-title"><span class="icon-sm">💡 </span> Intelligent
Recommendations</h3>
        <div class="recommendation-grid" id="recGrid"></div>
    `;
    targetContainer.appendChild(panel);
    const recGrid = document.getElementById('recGrid');
    if (window.DJANGO_VARS.recommendations &&
window.DJANGO_VARS.recommendations.length) {
        window.DJANGO_VARS.recommendations.forEach(rec => {
            const card = document.createElement('div');
            card.className = 'rec-card';
            card.innerHTML = `
                <div class="rec-header"><h4>Missing Column:
<span>${rec.column}</span></h4></div>
                <div class="rec-body">
                    <p>${rec.message}</p>
                    <div class="formula-box"><strong>Formula:</strong>
<code>${rec.formula}</code></div>
                </div>
            `;
            recGrid.appendChild(card);
        });
    } else {
        const noRecCard = document.createElement('div');
        noRecCard.className = 'rec-card';
        noRecCard.innerHTML = `
            <div class="rec-body"><p><strong>✓ Structure Verified:</strong> All
expected columns and formulas are present.</p></div>
        `;
        recGrid.appendChild(noRecCard);
    }

    captureState();
    console.log("📦 Editor initialized with multi-cell selection and advanced
formulas.");
    console.log("📦 STEP 5 (JS IN): Page loaded. What did Django give us?");
    console.log("Raw DJANGO_VARS.styleData type:", typeof
window.DJANGO_VARS.styleData);
    console.log("Raw DJANGO_VARS.styleData value:",
window.DJANGO_VARS.styleData);
    console.log("Parsed styles variable:", styles);
}
}
init();

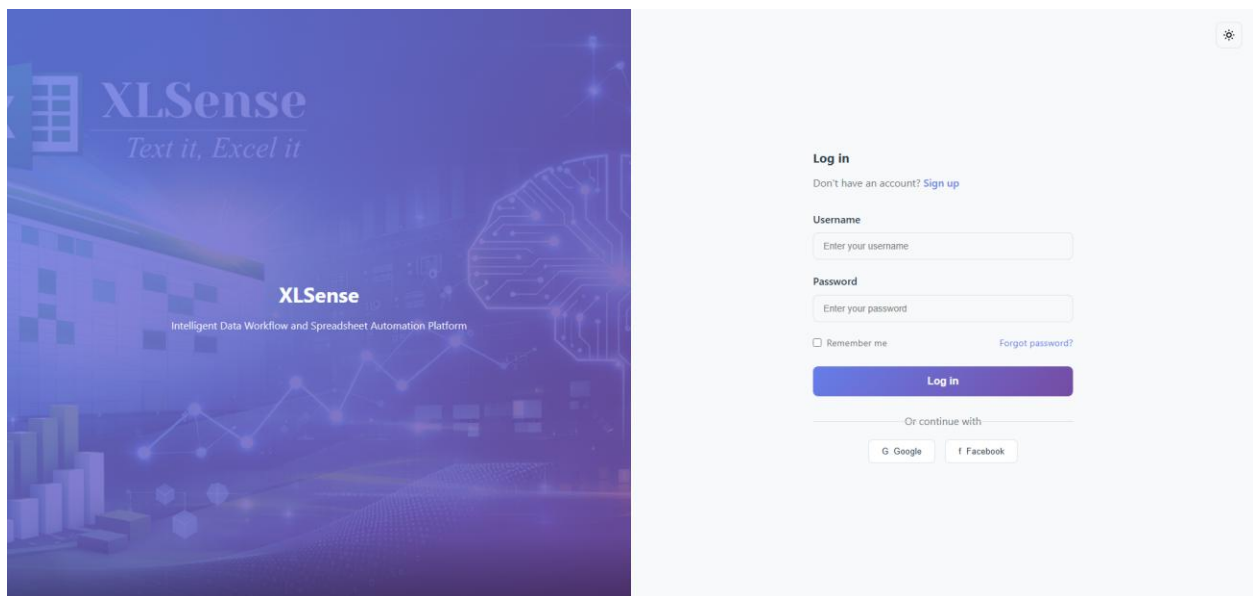
```

6.5 UI Snippet

Homepage:



Login:



Sign up:



XLSense
Text it, Excel it

XLSense
Intelligent Data Workflow and Spreadsheet Automation Platform

Sign up
Already have an account? [Log in](#)

Username
Choose a username
Required: 150 characters or fewer.

Email
Enter your email

Password
Create a password
Must be at least 8 characters.

Confirm Password
Confirm your password

☐ I agree to the [Terms of Service](#)

Create account

Or continue with

[G Google](#) [f Facebook](#)

User Dashboard:

XLSense

Dashboard

Domain Templates

Upload File

Workbook Editor

Upload History

Logout

My Dashboard

Total Files
4

Processed
4

Pending
0

File Upload History

Filename	Domain	Status	Upload Time	Action
TA_Excel_Practice_eKNIW2IW.xlsx	Academic	Completed	May 04, 2026 16:28	Download
TA_Excel_PracticeWrng_8yrMoVT.xlsx	Academic	Completed	May 04, 2026 16:28	Download
Inventory_bxt_doc_converted_juGHWmP.xlsx	Academic	Completed	May 04, 2026 16:28	Download
HR_PDF_converted_rSCmqCg.xlsx	Academic	Completed	May 04, 2026 16:28	Download

Admin Dashboard:

XLSense

Dashboard

Audit Logs

Domain Templates

Upload File

Workbook Editor

Upload History

Logout

My Dashboard

Welcome back, rubaiya!

Total Files

4

Processed

4

Pending

0

Administrative Control Panel

You have elevated privileges. Use the adjacent tools to monitor system health or manage raw data.

System Stats

DB Admin

File Upload History

Filename	Domain	Status	Upload Time	Action
TA_Excel_Practice_eKNIW2W.xlsx	Academic	Completed	May 04, 2026 16:28	Download
TA_Excel_PracticeWrng_8yrMoVT.xlsx	Academic	Completed	May 04, 2026 16:28	Download
Inventory_txt_doc_converted_juGHWmP.xlsx	Academic	Completed	May 04, 2026 16:28	Download

Admin Dashboard (System Stats):

XLSense

Dashboard

Audit Logs

Domain Templates

Upload File

Workbook Editor

Upload History

Logout

Admin Dashboard | XLSense

System Overview

Real-time stats for XLSense validation engine.

TOTAL USERS

7

System Users

TOTAL DOMAINS

4

Active in Database

FILES PROCESSED

4

System Wide

VALIDATION RULES

28

Core Logic Active

Recent System Activity

USER	ACTION TYPE	EVENT DETAILS	TIMELINE
rashefa	File Upload	Successfully uploaded and parsed: TA_Excel_Practice.xlsx Domain: Academic	3 minutes ago
rashefa	File Upload	Successfully uploaded and parsed: TA_Excel_Practice(Wrng).xlsx Domain: Academic	3 minutes ago
rashefa	File Upload	Converted Inventory txt doc.txt to structured Excel using AI Domain: Academic	3 minutes ago
rashefa	File Upload	Converted HR PDF.pdf to structured Excel using AI Domain: Academic	3 minutes ago

View All Logs

76

Audit Logs (Admin only):

XLsense

Dashboard

Audit Logs

Domain Templates

Upload File

Workbook Editor

Upload History

Logout

Audit Logs | XLSense

admin.rubaiyaADMINISTRATOR

Audit Logs140 total records

SEARCH

USER

ACTION TYPE

FROM DATE

TO DATE

Apply Filters

Clear

USER	ACTION	ENTITY	TIMESTAMP	DETAILS / METADATA
admin.rubaiya	Report View	Repo	Apr 24, 2026 02:04:20	Viewed validation report for uploads/academic_boundary.csv
admin.rubaiya	File Restored	File	Apr 24, 2026 02:03:58	Changed status to active: uploads/academic_boundary_converted_edited.xlsx
admin.rubaiya	File Restored	File	Apr 24, 2026 02:03:55	Changed status to active: uploads/academic_boundary_converted.xlsx
admin.rubaiya	File Restored	File	Apr 24, 2026 02:03:53	Changed status to active: uploads/academic_boundary.csv
admin.rubaiya	File Restored	File	Apr 24, 2026 02:03:52	Changed status to active: uploads/test_data_edited_88m1Ru.xlsx
admin.rubaiya	File Restored	File	Apr 24, 2026 02:03:50	Changed status to active: uploads/test_data_edited_edited.xlsx
admin.rubaiya	Delete	File	Apr 24, 2026 02:03:47	Changed status to user_deleted: uploads/test_data_edited_edited.xlsx
admin.rubaiya	Delete	File	Apr 24, 2026 02:03:46	Changed status to user_deleted: uploads/test_data_edited_88m1Ru.xlsx
admin.rubaiya	Delete	File	Apr 24, 2026	Changed status to user_deleted: uploads/academic_boundary.csv

Domain Templates:

XLsense

Dashboard

Audit Logs

Domain Templates

Upload File

Workbook Editor

Upload History

Logout

Domain Templates

admin.rubaiyaADMINISTRATOR

Domain TemplatesLibrary of validation logic and automated formulas.

SELECT DOMAIN

Academic

Hr

Finance

Inventory

Quick search rules or columns...

Validation Rules

RULE NAME	COLUMN	CONDITION
Score Range	Score	0 <= x <= 100
Attendance Check	Attendance	0 <= x <= 100
Bonus Cap	Bonus	0 <= x <= 20
Total Upper Bound	Total	x <= 120

Formula Rules

FORMULA NAME	TARGET	LOGIC
Total Calculation	Total	Score + Bonus
Grade Assignment	Grade	("A" if Total >= 90 else "B" if Total >= 75 else "C" if Total >= 60 else "D" if Total >= 40 else "F")

Upload Page:

XLSense

Dashboard

Audit Logs

Domain Templates

Upload File

Workbook Editor

Upload History

Logout

Overview

admin.rubaiya

ADMINISTRATOR

Upload Excel or CSV File

Select Domain:

Academic

Select File(s):

Choose File(s)

Upload File

Latest Uploads

academic_boundary_converted_edited.xlsx

academic_boundary_converted.xlsx

academic_boundary.csv

Upload Page (Column Detection):

XLSense

Dashboard

Audit Logs

Domain Templates

Upload File

Workbook Editor

Upload History

Logout

TA_Excel_Practice.xlsx

Detected Column Types

Column	Detected Type
serial_no.	Numeric
student_id	Numeric
name	Text
attendance_(10%)	Numeric
assignment_(10%)	Numeric
attendance_(5%)	Numeric
assignment_(5%)	Numeric
quiz_1	Numeric
quiz_2	Numeric
quiz_3	Numeric
quiz(2/3)(20%)	Numeric
mid_1	Numeric
mid_2	Numeric
mid(1/2)(30%)	Numeric
final_(40%)	Numeric

Upload Page (Spreadsheet Preview):

XLsense

Dashboard

Audit Logs

Domain Templates

Upload File

Workbook Editor

Upload History

Logout

Spreadsheet Preview

serial_no.	student_id	name	attendance_(10%)	assignment_(10%)	attendance_(5%)	assignment_(5%)	quiz_1	quiz_2	quiz_3	quiz(2/3) (20%)	mid_1	mid_2	mid(1/2) (30%)
1	221001	Rubaiya Akter	9	8	4.5	4.0	15	17	18	17.5	22	24	24
2	221002	Adita Haq	10	9	5.0	4.5	16	14	15	15.5	20	23	23
3	221003	Zarin Arefin Deya	8	7	4.0	3.5	12	13	14	13.5	18	21	21
4	221004	Fahim Rahman	7	6	3.5	3.0	10	11	12	11.5	15	18	18
5	221005	Sadia Islam	9	9	4.5	4.5	18	19	17	18.5	25	26	26
6	221006	Tanvir Ahmed	8	8	4.0	4.0	14	15	16	15.5	21	22	22
7	221007	Nusrat Jahan	10	10	5.0	5.0	19	18	20	19.5	27	28	28
8	221008	Arif Hossain	6	5	3.0	2.5	9	10	11	10.5	14	17	17
9	221009	Mehedi Hasan	7	7	3.5	3.5	13	12	14	13.5	19	20	20
10	221010	Samira Noor	9	8	4.5	4.0	17	16	18	17.5	24	25	25
11	221011	Imran Khan	8	7	4.0	3.5	15	14	13	14.5	20	21	21

Upload Page (Excel Preview):

Format as Tables: Default
Download Styled Excel Style: Default

Serial No.	Student Id	Name	Attendance (10%)	Assignment (10%)	Attendance (5%)	Assignment (5%)	Quiz 1	Quiz 2	Quiz 3	Quiz(2/3)(20%)	Mid 1	Mid 2
1	221001	Rubaiya Akter	9	8	4.5	4.0	15	17	18	17.5	22	24
2	221002	Adita Haq	10	9	5.0	4.5	16	14	15	15.5	20	23
3	221003	Zarin Arefin Deya	8	7	4.0	3.5	12	13	14	13.5	18	21
4	221004	Fahim Rahman	7	6	3.5	3.0	10	11	12	11.5	15	18
5	221005	Sadia Islam	9	9	4.5	4.5	18	19	17	18.5	25	26
6	221006	Tanvir Ahmed	8	8	4.0	4.0	14	15	16	15.5	21	22
7	221007	Nusrat Jahan	10	10	5.0	5.0	19	18	20	19.5	27	28
8	221008	Arif Hossain	6	5	3.0	2.5	9	10	11	10.5	14	17
9	221009	Mehedi Hasan	7	7	3.5	3.5	13	12	14	13.5	19	20
10	221010	Samira Noor	9	8	4.5	4.0	17	16	18	17.5	24	25
11	221011	Imran Khan	8	7	4.0	3.5	15	14	13	14.5	20	21
12	221012	Rafiqul Islam	6	6	3.0	3.0	11	12	10	11.5	16	18
13	221013	Tanjila Akter	10	9	5.0	4.5	20	19	18	19.5	28	27
14	221014	Shafin Mahmud	7	8	3.5	4.0	14	13	15	14.5	21	22

Workbook editor list page:

XLsense

Dashboard

Audit Logs

Domain Templates

Upload File

Workbook Editor

Upload History

Logout

Overview

Workbook Editor

Select a file to begin domain mapping and data cleaning.

Original Uploads

Needs Mapping

Files that haven't been mapped to database fields yet.

FILENAME	DOMAIN	STATUS	UPLOAD TIME	ACTION
TA_Excel_PracticeWrng_3HwuNIG.xlsx	Academic	Completed	May 04, 2026 16:54	Open Editor
TA_Excel_Practice_D0nkZy.xlsx	Academic	Completed	May 04, 2026 16:35	Open Editor

Edited Workbooks

Saved & Mapped

Files with completed mapping and processing.

FILENAME	DOMAIN	STATUS	LAST MODIFIED	ACTION
TA_Excel_PracticeWrng_3HwuNIG_edited.xlsx	Academic	Completed	May 04, 2026 16:55	View Edited File

Workbook editor page (Error cells & AI Explanation):

uploads/TA_Excel_PracticeWrng_RXzbr2j.xlsx

UndoRedoAdd RowAdd ColumnSave & Process

Enter formula (e.g., =SUM(A1:A5))

FXQ3Formulas

CAI Explain

2/3(20%)	MID_1	MID_2	MID(1/2)(30%)	FINAL_(40%)	TOTAL	GRADE	STATUS	ENROLLED	BONUS	FINAL GRADE
	Mid	Mid	Map to DB field	Final	Total Marks	Grade	Status	Enrolled	Bonus	Final Grade
1		24	24	35	85.0	B				
2		23	23	32	82.0	B-				
3		21	21	30	72.0	C				
4		18	18	25	61.0	A				
5		26	26	38	91.5	A-				
6		22	22	33	78.5	C				
7		28	28	39	96.5	A				
8		17	17	24	57.0	C				
9		20	20	29	69.5	C				
10		25	25	36	87.0	B+				
11		21	21	31	74.0	C				
12		18	18	26	61.5	C				
13		27	28	40	97.0	A				
14		22	22	34	78.0	C				
15		27	27	38	92.5	A-				

Intelligent Auditor Analysis

Explanation of that validation error:

- The error occurs in a grade calculation formula.
- The formula checks the "total" score and assigns a letter grade based on specific ranges.
- The conditions and corresponding grades are:
 - "A" if total score is 93 or above
 - "A-" if total score is 90-92
 - "B+" if total score is 87-89
 - "B" if total score is 83-86
 - "B-" if total score is 80-82
 - "C+" if total score is 79
 - "C" if total score is 76-78
 - "C-" if total score is below 76
- The issue likely arises from inconsistent or incorrect total scores, causing an invalid grade to be assigned.

Workbook editor page (Live Error Validations):

uploads/TA_Excel_PracticeWmg_RXzbr2j.xlsx

UndoRedoAdd RowAdd ColumnSave & Process

Inter14pxB I U A

-COUNTIF(Q1:Q30, "A")

FXQ31Formulas

-COUNTIF(range, "criteria") | Example: =COUNTIF(A1:A30, ">90")

7

QUIZ_2	QUIZ_3	QUIZ(2/3)(20%)	MID_1	MID_2	MID(1/2)(30%)	FINAL_(40%)	TOTAL	GRADE	STATUS	ENROLLED
Quiz	Quiz	Mid	Mid	Mid	Final	Total Marks	Grade	Status	Enrolled	
21	16.5	23	24	24	36	85.0	B			
22	50.5	14	16	16	22	94.0	A	Passed		
23	19.5	28	29	29	40	98.0	A			
24	12.5	17	18	18	26	63.0	F		Enrolled	
25	15.5	21	23	23	33	79.0	C+	Passed		
26	17.5	25	26	26	37	89.5	B+	Passed	Dropped	
27	11.5	16	17	17	25	59.5	F			
28	19.5	28	29	29	40	98.5	A	Withdrawn	Withdrawn	
29	13.5	18	20	20	28	68.5	F			
30	17.5	24	25	25	36	97.0	A	Passed	Registered	
31				22.5	22	98.5	7			

31 rows · 19 columnsReady · XLSense Intelligent Automation

Workbook editor page (Formula Recommendations):

uploads/TA_Excel_PracticeWmg_RXzbr2j.xlsx

UndoRedoAdd RowAdd ColumnSave & Process

Inter14pxB I U A

Enter formula (e.g., =SUM(A1:A5))

FXC31Formulas

30

SERIAL_NO.	STUDENT_ID	NAME	ATTENDANCE_(10%)	ASSIGNMENT_(10%)	ATTENDANCE_(5%)	ASSIGNMENT_(5%)	QUIZ_1	QUIZ_2	QUIZ_3
Sl No	Student Id	Name	Attendance (10%)	Assignment (10%)	Attendance (5%)	Assignment (5%)	Quiz	Quiz	Quiz

Intelligent Recommendations

MISSING COLUMN: BONUS

Suggested: Create 'Bonus' column (required for validation).

Formula: None (Manual Data Entry)

MISSING COLUMN: STATUS

Suggested: Create 'Status' column. Use the formula below to calculate it.

Formula: 'Passed' if Total >= 60 else 'Failed'

MISSING COLUMN: ENROLLED

Suggested: Create 'Enrolled' column (required for validation).

Formula: None (Manual Data Entry)

MISSING COLUMN: FINAL GRADE

Suggested: Create 'Final Grade' column (required for validation).

Formula: None (Manual Data Entry)

31 rows · 19 columnsReady · XLSense Intelligent Automation

Upload history:

XLSense

Dashboard

Audit Logs

Domain Templates

Upload File

Workbook Editor

Upload History

Logout

Overview

admin.rubaiya

ADMINISTRATOR

File Upload History

Total Records: 57

Trash

Upload New File

USER	FILENAME	DOMAIN	STATUS	DATE & TIME	ACTIONS
admin.rubaiya	academic_boundary_converted_edited.xlsx	Academic	Completed	Apr 24, 2026 00:54	View Report Delete
admin.rubaiya	academic_boundary_converted.xlsx	Academic	Completed	Apr 24, 2026 00:53	View Report Delete
admin.rubaiya	academic_boundary.csv	Academic	Completed	Apr 24, 2026 00:28	View Report Delete
admin.rubaiya	test_data_edited_f88m1Ru.xlsx	Academic	Completed	Apr 24, 2026 00:04	View Report Delete
admin.rubaiya	test_data_edited_edited.xlsx	Academic	Completed	Apr 24, 2026 00:04	View Report Delete

Upload history (Trash):

XLSense

Dashboard

Audit Logs

Domain Templates

Upload File

Workbook Editor

Upload History

Logout

Overview

rubaiya

ADMINISTRATOR

Trash

Files in trash: 4

Back to History

FILENAME	DOMAIN	DELETED	EXPIRES	ACTIONS
TA_Excel_Practice_DXnkZy.xlsx	Academic	May 04, 2026 17:06	Jun 03, 2026	Restore
TA_Excel_PracticeWrng_3HwuNIG_edited.xlsx	Academic	May 04, 2026 17:05	Jun 03, 2026	Restore
HR_PDF_converted_Wvj8OIN.xlsx	Academic	May 04, 2026 16:54	Jun 03, 2026	Restore
Inventory_txt_doc_converted_YnDzWUA.xlsx	Academic	May 04, 2026 16:54	Jun 03, 2026	Restore

Report & AI implementation in upload history:

XLsense
Dashboard
Audit Logs
Domain Templates
Upload File
Workbook Editor
Upload History

Logout

Validation Report
File: TA_Excel_PracticeWrg_3HwuNIG_edited.xlsx

Back to History

DATA QUALITY SCORE

40.00%

This score represents the percentage of rows that passed all validation rules.

Issues Detected
Found 19 validation errors that require attention.

ROW	FIELD/COLUMN	ERROR DESCRIPTION	STATUS
2	attendance_(5%)	Attendance cannot exceed 5%.	Failed
INTELLIGENT AUDITOR ANALYSIS Intelligent Auditor Analysis Explanation of that validation error: - The validation rule is checking the value in the 'attendance_(5%)' column. - The condition is that attendance should not be more than 5%. - If the attendance value entered exceeds 5%, the validation error occurs. - This error means the entered attendance percentage is higher than the allowed 5% limit.			
3	grade	Manual Entry Error: The Letter Grade in this row does not match the required threshold for the calculated Total marks.	Failed
4	grade	Manual Entry Error: The Letter Grade in this row does not match the required threshold for the calculated Total marks.	Failed

Formula Recommendations in Upload Page:

XLsense
Dashboard
Audit Logs
Domain Templates
Upload File
Workbook Editor
Upload History

Logout

- To fix the error:
- Ensure that the "total" value is a valid number.
- Check that the "total" value is within the expected range (0-100).

30 Final (40%) **Final marks must be between 0-40.** Failed

AI Explain

INTELLIGENT AUDITOR ANALYSIS
Intelligent Auditor Analysis
Explanation of that validation error:

- The validation rule is set to ensure that the "Final" marks are within a specific range.
- The allowed range for "Final" marks is between 0 and 40 (inclusive).
- The error occurs when the entered "Final" marks are outside this range, i.e., either less than 0 or greater than 40.
- This rule is in place to ensure that the final marks are within the specified 40% range.

Intelligent Recommendations

MISSING COLUMN: **BONUS**
Suggested: Create 'Bonus' column (required for validation).
Formula: `None (Manual Data Entry)`

MISSING COLUMN: **STATUS**
Suggested: Create 'Status' column. Use the formula below to calculate it.
Formula: `'Passed' if Total >= 60 else 'Failed'`

MISSING COLUMN: **FINAL GRADE**
Suggested: Create 'Final Grade' column (required for validation).
Formula: `None (Manual Data Entry)`

MISSING COLUMN: **ENROLLED**
Suggested: Create 'Enrolled' column (required for validation).
Formula: `None (Manual Data Entry)`

6.6 Budget Planning:

Component	Estimated Cost (BDT)	Notes
Domain/Hosting (Cloud)	3,000	Annual hosting for deployment
Development Machine (if new)	80,000	Laptop for development
API Calls (AI)	1,500	Development quota for OpenAI-compatible API
Documentation & Printing	500	Report printing and binding
Miscellaneous	1,000	Contingency
Total	86,000 BDT (~800 USD)	Primarily development hardware

Chapter 7: Conclusion

7.1 Summary

XLSense is a comprehensive, Django-based web application that automates spreadsheet validation, formula auditing, and AI-powered data processing. The platform supports four domain-specific templates (Academic, HR, Finance, Inventory) with over 20 validation rules and 15 formula recommendations per domain. Users can upload files in multiple formats (.xlsx, .csv, .pdf, .txt), receive validated outputs with quality scores, access AI-powered plain-language explanations, and edit spreadsheets in a live workbook editor with real-time cell validation. The system includes role-based access control, a comprehensive audit trail, soft delete with 30-day expiry, and three-theme UI support (light/medium/dark).

Built with modern Python tools — Django 6.0, pandas 2.3.3, openpyxl 3.1.5 — and tested with pytest and Playwright, XLSense demonstrates practical software engineering skills including full-stack development, API integration, data science tooling, and collaborative team-based development. The platform fills a genuine research gap by combining domain-specific rule engines, AI explanations, and text-to-spreadsheet conversion in a single open-source application.

7.2 Limitations

1. **AI Dependency:** Full functionality requires a valid OpenAI-compatible API key. Without it, text-to-spreadsheet conversion falls back to a basic whitespace parser, and formula explanations use simple rule-based responses.
2. **PDF OCR:** XLSense cannot process scanned PDFs without selectable text, as PyPDF2 cannot extract text from image-based PDFs.
3. **File Size Limits:** Large files may timeout during AI conversion due to API token limits and processing time constraints.
4. **Domain Scope:** Only four predefined domains are supported. Adding custom domains requires database entries via Django admin, with no UI-based custom domain builder.
5. **Single-User Sessions:** No real-time collaborative editing. Multiple users editing the same file simultaneously could overwrite each other's changes.
6. **No Mobile Optimization:** The web interface is not mobile-responsive, limiting accessibility on mobile devices.

7.3 Future Improvement

1. **WebSocket-Based Collaborative Editor:** Enable real-time multi-user editing using Django Channels or a WebSocket server, allowing simultaneous collaboration on spreadsheets.

2. **Mobile Application:** Develop a Flutter or React Native mobile app for on-the-go file validation and viewing.
3. **Custom Domain Builder:** Provide a UI for users to define their own domain templates, validation rules, and formula recommendations without touching Django admin.
4. **Scheduled Processing with Celery:** Implement background task queuing for large file processing, with email notifications on completion.
5. **Analytics Dashboard:** Add charts and trend visualizations showing validation success rates, common errors, and user activity over time.
6. **Bulk Export:** Enable batch downloading of multiple processed files as a ZIP archive.
7. **Browser Extension:** Develop a browser extension for Google Sheets that provides direct validation and formula suggestions.

References

- [1] Wakefield Research, "Spreadsheet Risk Survey," 2023. [Online]. Available: <https://www.wakefieldresearch.com>
- [2] R. Panko, "What We Don't Know About Spreadsheet Errors," *Journal of End User Computing*, vol. 10, no. 3, pp. 48–54, 1998.
- [3] W. Chen et al., "Large Language Models for Unstructured Data Extraction," *Int. Conf. on AI and Big Data*, Chengdu, China, pp. 326–331, 2022.
- [4] Django Software Foundation, "Django Documentation," 2024. [Online]. Available: <https://docs.djangoproject.com/>
- [5] Pandas Development Team, "pandas: Powerful Python Data Analysis Toolkit," 2024. [Online]. Available: <https://pandas.pydata.org/>
- [6] OpenAI, "OpenAI API Documentation," 2024. [Online]. Available: <https://platform.openai.com/docs/>
- [7] H. Mohanty and S. S. V. N. D. P., "Automated Spreadsheet Error Detection Using Machine Learning," *IEEE Access*, vol. 8, pp. 182345–182356, 2020.
- [8] M. J. B. W. Boge, "A Systematic Review of Spreadsheet Quality Assurance," *Empirical Software Engineering*, vol. 24, pp. 3128–3168, 2019.